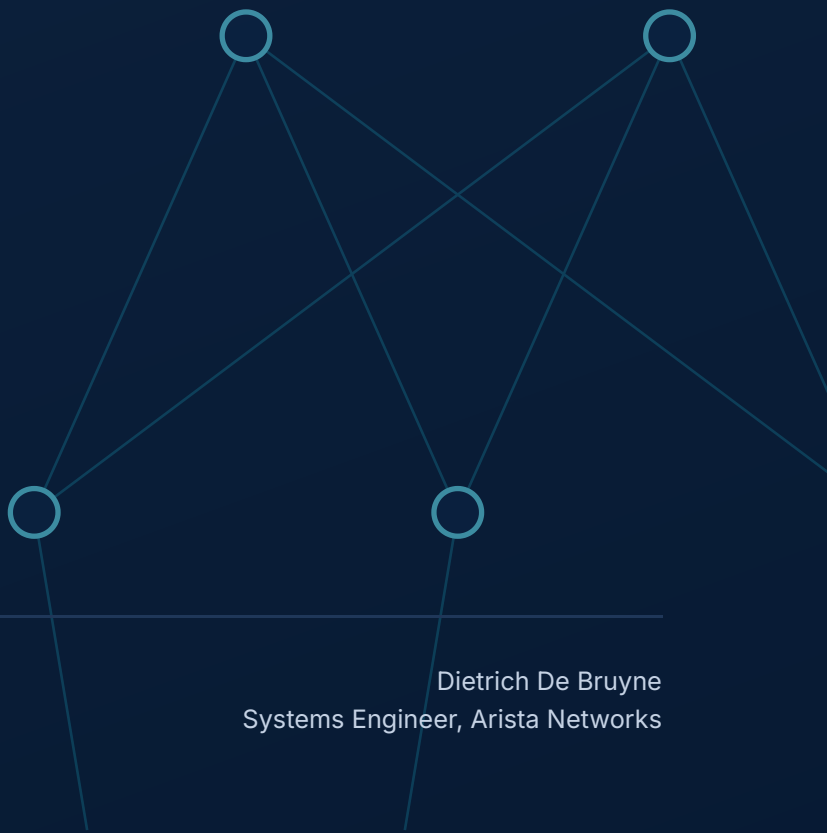
avdlab
AVD · ANTA · CONTAINERLAB

Security release 1.1

Security architecture, policies, operating procedures and the compliance matrix (NIST SSDF, ISO/IEC 27001:2022, SOC 2) for operators and reviewers.

- Roles, MFA policy, server-side sessions
- Encrypted secrets with ownership and rotation
- Audit trail, redaction, retention
- Hardened containers, pinned supply chain
- Signed releases, SBOM, provenance
- SSDF · ISO 27001 · SOC 2 matrix



v1.1.0

22 October 2026

Dietrich De Bruyne
Systems Engineer, Arista Networks

Contents

Ch. 0 **Overview** 3

Who it is for 3
How it relates to the other documents 3
The documents 3
Assurance summary 4
Versions and maintenance 4

Ch. 1 **Security architecture** 5

1.1 Trust zones 5
1.2 Boundary crossings and controls 7
1.3 Authentication and session design 7
1.4 Authorisation model 9
1.5 Secret management 9
1.6 External services: what each one sees 10
1.7 Transport security 11
1.8 Container and host hardening 11
1.9 Self-update supply chain 11
1.10 Job isolation, redaction and limits 12
1.11 Logging and audit design 12
1.12 What is deliberately not protected 12

Ch. 2 **Data classification** 13

2.1 Classes 13
2.2 Inventory 13
2.3 Encryption summary 14
2.4 Handling rules per class 15
2.5 Data flows that leave the deployment 15
2.6 Deletion and retention controls 15
2.7 Data location 16

Ch. 3 **Risk register** 17

3.1 Rating scale 17
3.2 Register 17
3.3 Known limitations at 1.1.0 (summary) 19
3.4 Review procedure 19
3.5 Coverage by trust zone and STRIDE 19
3.6 Treatment plan template 20

Ch. 4 **Access control policy** 21

4.1 Roles 21
4.2 Account lifecycle 21
4.3 Authentication policy 22
4.4 Multi-factor authentication 22
4.5 Session policy 22
4.6 Automation and API access 23
4.7 Least privilege on lab hosts 23
4.8 Secrets ownership rules 23
4.9 Periodic access review (quarterly) 23
4.10 Offboarding checklist 24

Ch. 5 **Secure development** 25

5.1 Pipeline overview 25
5.2 Source control and change management 27
5.3 Static analysis 27
5.4 Software composition analysis and dependency pinning 27
5.5 Secret scanning 28

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

5.6 Tests 28
5.7 Release process and verification 28
5.8 Vulnerability management 29
5.9 Third parties and suppliers 29
5.10 Secure coding conventions 30
5.11 Known SDLC gaps at 1.1.0 30

Ch. 6 Logging & monitoring 32
6.1 Log inventory 32
6.2 Audit events 33
6.3 What to alert on 33
6.4 Export and forwarding 34
6.5 Review cadence 35
6.6 Clock synchronisation 35

Ch. 7 Incident response 36
7.1 Roles 38
7.2 Severity 38
7.3 Triage (first 30 minutes) 38
7.4 Scenario: compromised user or admin account 38
7.5 Scenario: leaked AVDLAB_SESSION_SECRET 39
7.6 Scenario: leaked Fernet key (AVDLAB_SECRET_KEYS / /data/secret.key)
7.7 Scenario: leaked lab-host SSH key or password 39
7.8 Scenario: compromised lab host 39
7.9 Scenario: malicious or unexpected self-update / image 40
7.10 Scenario: suspected data exposure in job logs 40
7.11 Scenario: vulnerable dependency or product vulnerability 40
7.12 Eradication, recovery and post-incident review 40
7.13 Notification considerations 41
7.14 Exercise 41

Ch. 8 Backup & recovery 42
8.1 What to back up 42
8.2 Consistency 42
8.3 Encryption and storage requirement 43
8.4 Schedule example 43
8.5 RPO / RTO statement (template) 43
8.6 Restore procedure 44
8.7 Quarterly restore test checklist 44
8.8 Consequences of key loss 44

Ch. 9 Hardening checklist 46
9.1 Before you start 46
9.2 Environment (deploy/.env)
9.3 Network exposure and profiles 46
9.4 Container hardening (verify, do not change) 47
9.5 Caddy, TLS and browser headers 47
9.6 Accounts, MFA and secrets 47
9.7 Updates and supply chain 47
9.8 Lab hosts 48
9.9 Backups, logging, monitoring 48
9.10 Deployment record 48

Ch. 10 Compliance matrix 49
10.1 NIST SSDF 1.1 49
10.2 ISO/IEC 27001:2022 Annex A 50
10.3 SOC 2 Trust Services Criteria 52
10.4 Using the matrix 52

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

CHAPTER 0

Overview

What this set is, who it is for, and the assurance summary per framework.

This is the **security documentation set** for avdlab 1.1.0, the web-administered Arista AVD/ANTA lab orchestrator built in this repository (FastAPI backend, React SPA, Docker deployment, lab hosts reached over SSH). It describes the security architecture of the product, the policies and procedures an operator needs to run it responsibly, and how the product's controls map to the frameworks a reviewer will ask about. Everything in it was written against the code at version `1.1.0` and the implementation notes of the three hardening streams (`docs/_changes/A.md`, `B.md`, `C.md`); where a statement names a file, a setting or a command, that is where the behaviour lives.

Who it is for

READER	START WITH	THEN
Operator who installs and runs avdlab	09-hardening-checklist.md	04 (access control), 06 (logging), 08 (backups), 07 (incident response)
Security reviewer / auditor (ISO/IEC 27001:2022, SOC 2, NIST SSDF 1.1)	01-security-architecture.md	03 (risk register), 10 (compliance matrix), 05 (secure development), 02 (data classification)
Developer / maintainer	05-secure-development.md	01 §1.12 (what is deliberately not protected), 03 (residual risks)

How it relates to the other documents

- `docs/1.1/` is the **product documentation**: how avdlab works and how to use it. Its chapter 2 §(f) *Security model* describes what the code does (authentication, sessions, RoleGuard, secret store, redaction, headers, supply chain). This set does not repeat that description line by line; it references it and adds the trust boundaries, the data classification, the risks, the policies, the procedures and the framework mapping around it. Chapter 3 §3.5 of the product documentation is the authoritative settings reference; chapter 8 §8.6 the key-rotation procedure; chapter 9 §9.7 the secure-coding conventions.
- `SECURITY.md` is the public **vulnerability disclosure policy**: supported versions, how to report, response targets (acknowledgement within 3 business days, fix targets 30/90 days), scope, by-design behaviours and safe harbour. The procedures here (05 §5.8, 07) implement it.
- `CONTRIBUTING.md` is the **contributor-facing SDLC**: branching, required checks, lockfiles, release procedure. Chapter 05 is the assurance view of the same process.
- `CHANGELOG.md` is the feature and fix history; security fixes appear under *Security* with the advisory id once one exists.

The documents

#	FILE	WHAT IT ANSWERS
—	README.md	this index and the assurance summary
1	01-security-architecture.md	trust zones and boundaries, data flows, authentication and session design, authorisation, secret management and where every credential travels, transport security, container hardening, self-update supply chain, job isolation and redaction, logging design, what is deliberately not protected
2	02-data-classification.md	asset and data inventory: six data classes, where each is stored, encryption at rest and in transit, who can read it, retention, backup inclusion, handling rules
3	03-risk-register.md	STRIDE-style risk register over the trust-zone diagram: ~20 risks with existing controls, residual risk, owner and review date; the known limitations stated honestly
4	04-access-control-policy.md	roles and what each may do, account lifecycle, MFA requirement, password and session policy, periodic access review, least privilege on lab hosts, secrets ownership, offboarding
5	05-secure-development.md	the SDLC as practised: branch protection, review checklist, static analysis, SCA, secret and container scanning, tests, the signed release process and how a consumer verifies it, dependency pinning, suppliers, vulnerability management SLAs, secure-coding conventions
6	06-logging-monitoring.md	log inventory, sensitivity and redaction, retention, export and forwarding, what to alert on, review cadence, clock synchronisation
7	07-incident-response.md	the playbook: roles, severity, triage, containment with exact commands per scenario, recovery, post-incident review template, notification
8	08-backup-recovery.md	what to back up, schedule example, encryption requirement, RPO/RTO template, restore procedure, quarterly restore test, consequences of key loss
9	09-hardening-checklist.md	tick-box deployment checklist for LAN and public deployments, lab hosts, NetBox/Postgres, update verification, secrets review
10	10-compliance-matrix.md	NIST SSDF 1.1 practices, ISO/IEC 27001:2022 Annex A controls and SOC 2 Trust Services Criteria mapped to product controls, evidence and operator responsibilities
—	`diagrams/`	Mermaid sources (`.mmd`) and rendered SVGs: `trust-zones`, `session-lifecycle`, `release-pipeline`, `incident-flow`

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

Conventions are the same as in `docs/1.1/`: `monospace` is a literal (a file path, a setting, an API path, a command); "orchestrator" is the machine running the avdlab container; "lab host" is the Linux machine (or VM) running containerlab and the cEOS containers; "operator" in a policy context is the organisation that runs a deployment, not the `operator` role unless written in monospace.

Assurance summary

A product cannot be certified; an organisation and its management system can. avdlab ships technical controls, evidence of how they are built and tested, and procedures an operator can adopt. ISO/IEC 27001 certification, a SOC 2 report and conformance to the NIST Secure Software Development Framework are statements about the **operating organisation** (its scope, risk treatment, management review, HR processes, supplier management and the evidence it retains). The three paragraphs below say, per framework, what avdlab provides and what remains the organisation's responsibility. The full mapping is in 10-compliance-matrix.md.

NIST SSDF 1.1 (SP 800-218). The repository practises the framework's four groups for a small project: the security requirements are written down (`SECURITY.md`, `CONTRIBUTING.md`, the PR security checklist, chapter 9 §9.7 conventions), roles are assigned through `CODEOWNERS`, the toolchain is pinned (base images by digest, GitHub Actions by commit SHA, Python by hash in `backend/requirements.lock`, npm by `package-lock.json`), code is reviewed and scanned (CodeQL `security-extended`, ruff with security rules, eslint, pip-audit, npm audit, gitleaks, Trivy), the release is built in CI from a tag that is never moved, with a CycloneDX SBOM, SLSA provenance and a keyless cosign signature that the consumer — and the product's own self-update — can verify, and vulnerabilities have a disclosure channel with response targets. **The organisation** remains responsible for keeping those checks required on `main` (branch protection is a GitHub setting, not a file), for running the suppliers' and its own vulnerability intake, for training the maintainers, and for documenting deviations (a `ruff format` backlog and dev-only npm advisories are known at 1.1.0).

ISO/IEC 27001:2022. avdlab implements or directly supports the technological controls of Annex A that apply to a web application and its deployment: access control with roles and MFA (5.15–5.18, 8.2, 8.5), secure authentication and server-side sessions (8.5), cryptography for secrets at rest and TLS in transit (8.24), logging with an actor, retention and export (8.15, 8.16), configuration and hardening defaults (8.9), secure coding and testing (8.25–8.29, 8.31), change management through pull requests (8.32), information deletion and retention settings (8.10, 8.13 via backups), and supply-chain controls for the components it ships (5.19–5.21, 8.19). **The organisation** owns the management system: scope, policies approved by management, risk assessment and treatment (this set's chapter 03 is a starting register, not the organisation's), asset ownership (5.9), HR screening and awareness (6.1–6.3), supplier agreements, backup execution and restore tests (8.13 is a procedure here; the organisation must run it), incident management (5.24–5.28 are a playbook here; the organisation must staff it), legal and privacy obligations (5.31–5.34), and the internal audit and management review that make the ISMS certifiable.

SOC 2 (Trust Services Criteria). avdlab provides the technical substance for the Common Criteria most relevant to a software system: logical access (CC6.1–CC6.3 — roles, MFA policy, session revocation, secret scoping, origin guard), protection of credentials at rest and in transit (CC6.1, CC6.6, CC6.7), change management with review and automated gates (CC8.1), system monitoring through an audit trail with export (CC7.2), vulnerability identification through CI scanning and Dependabot (CC7.1), incident response procedures (CC7.3–CC7.5), and availability through resource limits, health checks and documented backups (A1.2, A1.3). **The organisation** owns the control environment (CC1: integrity, board oversight, competence), communication (CC2), risk assessment (CC3), monitoring of controls (CC4), control activities as *performed* (CC5), vendor management (CC9.2), and the evidence of operation over the audit period (access reviews actually held, backups actually tested, incidents actually handled). A SOC 2 report is about the service organisation; a self-hosted avdlab appears in it as an in-scope system whose controls are the operator's.

Versions and maintenance

This set is versioned with the product. Each chapter states the version it describes in its first paragraph; when a later release changes a control, the chapter is updated in the same pull request (`PULL_REQUEST_TEMPLATE.md` → *Docs updated*). Review dates in the risk register (03) and the checklists (04 §4.9, 08 §8.7) are placeholders for the operating organisation to fill in. Corrections and gaps: open an issue, or for anything exploitable follow `SECURITY.md`.

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

CHAPTER 1

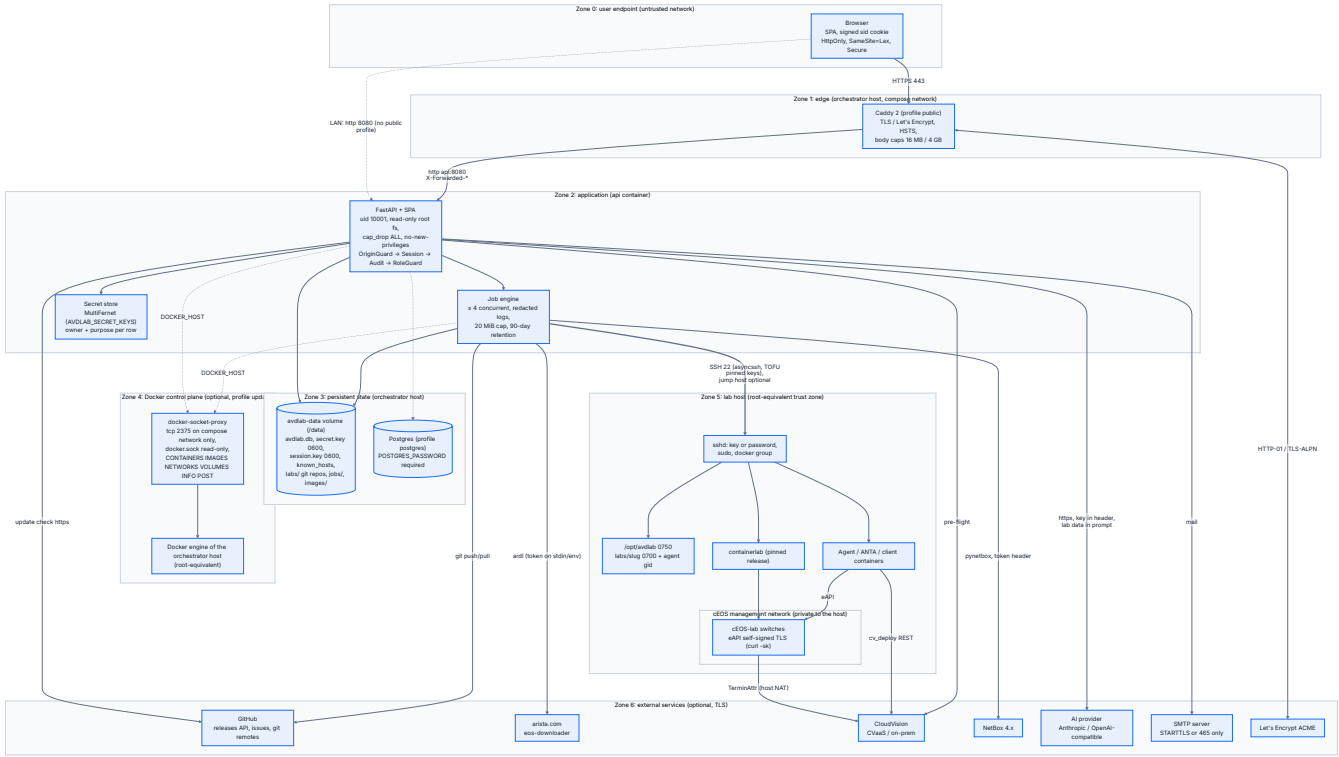
Security architecture

Trust zones, data flows, authentication, authorization, secrets, transport, containers, supply chain, by-design limits.

This chapter is the security view of avdlab 1.1.0: which trust zones exist, what crosses each boundary, and which control stands at every crossing. It complements `docs/1.1/02-architecture.md` §(f), which describes the mechanisms in the code; here the emphasis is on boundaries, assumptions and what is deliberately left unprotected. File and setting names are the evidence: a reviewer can open the file and confirm the claim.

1.1 Trust zones

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks



Trust zones and data flows

ZONE	CONTAINS	TRUST ASSUMPTION	WHO ADMINISTERS IT
0 — user endpoint	the browser, the user's network	untrusted; everything it sends is validated; it holds only a signed cookie carrying a session id	the user
1 — edge	Caddy (compose profile `public`), the published port	trusted to terminate TLS and set `X-Forwarded-*`; the API trusts those headers only from `AVDLAB_TRUSTED_PROXIES`	the operator
2 — application	the `api` container: FastAPI, SPA, job engine, secret store	trusted code; runs with the least privilege compose can give (uid 10001, read-only root fs, no capabilities, no Docker socket)	the operator (deploys), the maintainers (code)
3 — persistent state	the `avdlab-data` volume, optionally Postgres	holds every secret ciphertext **and**, by default, the key files; anyone with root on the orchestrator host can read everything	the operator
4 — Docker control plane	`docker-socket-proxy`, the host's Docker engine	root-equivalent on the orchestrator host; reachable from the api only through the filtered proxy, and only when the `update` profile is started	the operator
5 — lab host	sshd, sudo, Docker, containerlab, cEOS, agent containers	**root-equivalent trust zone**: avdlab runs `sudo containerlab` and `docker run` there; whoever can register a host or bind its credentials can run commands as root on it	the operator (provisions), operators (use)
6 — external services	GitHub, arista.com, CloudVision, NetBox, AI providers, SMTP, Let's Encrypt	third parties reached over TLS with a credential from the secret store; they see exactly the data listed in §1.6	the respective provider

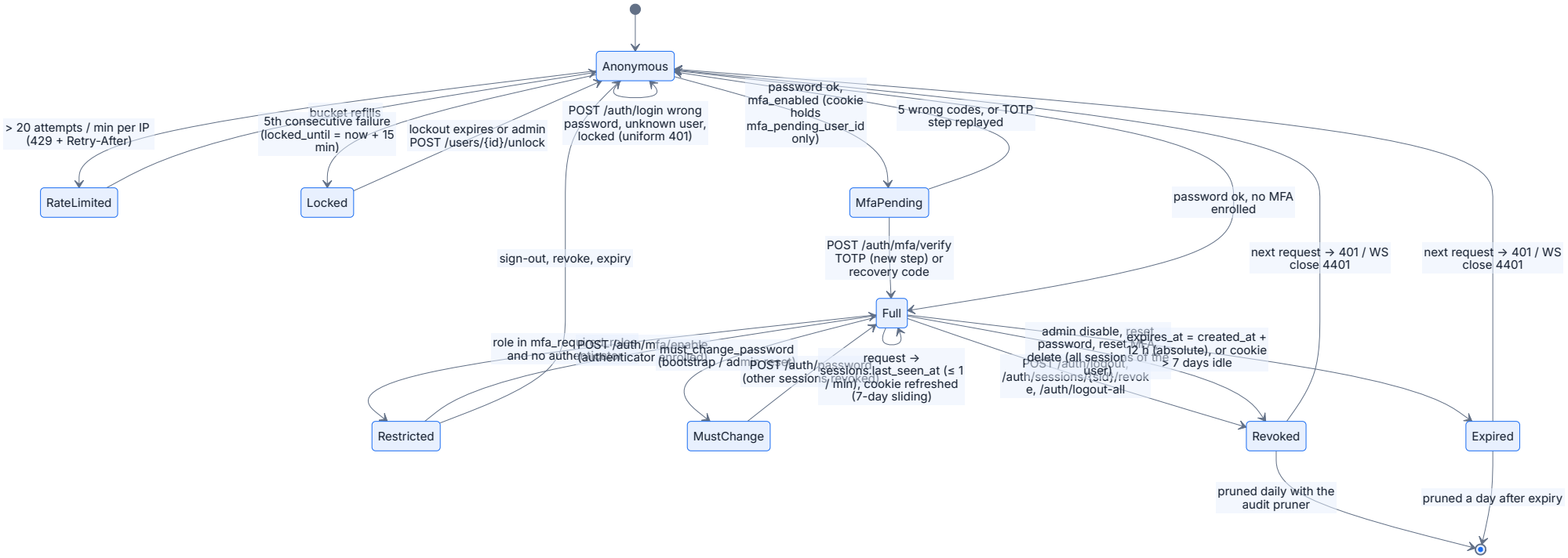
Two things the diagram makes explicit. First, **zone 2 is not a privilege boundary against zone 3**: the Fernet and session keys live in `/data` unless the operator moves them to the environment, so the data volume must be protected as if it held plaintext (chapter 02). Second, **zone 5 is deliberately trusted**: avdlab is an orchestrator for lab hosts the operator owns; it does not try to sandbox its own SSH commands (SECURITY.md → *By design*).

1.2 Boundary crossings and controls

CROSSING	CONTROL AT THE BOUNDARY	WHERE
browser → api (HTTP)	TLS by Caddy (public) or a trusted LAN; `OriginGuard` on every mutating `/api` request and WebSocket upgrade; `SessionMiddleware` (signed cookie, HttpOnly, `SameSite=Lax`, `Secure` behind https); `RoleGuard` (session row alive, MFA policy, role); per-IP token bucket on `/auth/login` and `/auth/mfa/verify`; response headers (CSP, Permissions-Policy, nosniff, DENY, Referrer-Policy)	`core/security.py`, `core/ratelimit.py`, `main.py`
Caddy → api	`ProxyHeadersMiddleware` only with `AVDLAB_TRUST_PROXY_HEADERS=true`, trusting `AVDLAB_TRUSTED_PROXIES` (never `*`); api bound to `127.0.0.1` with the public profile	`main.py`, `deploy/docker-compose.yml`, `deploy/Caddyfile`
api → data volume	the only writable mount; secret values Fernet-encrypted; key files 0600; container runs as 10001	`core/security.py::_load_keys`, `resolve_session_secret`, compose `read_only: true`
api → Docker engine	no socket in the container; `AVDLAB_DOCKER_HOST=tcp://docker-socket-proxy:2375`; proxy allows CONTAINERS/IMAGES/POST/NETWORKS/VOLUMES/INFO, BUILD/SESSION only with `AVDLAB_PROXY_ALLOW_BUILD=1`, socket mounted read-only, no published port	compose service `docker-socket-proxy`, `meta/update.py::helper_argv`
api → lab host (SSH)	asyncssh with trust-on-first-use host keys pinned in `/data/known_hosts` (jump hosts too), key or password from the secret store, `sudo -S` with the password on stdin, every revealed value registered with `ctx.redact()`; `jump_host` syntax validated	`hosts/ssh.py`, `hosts/recipes.py`
lab host → cEOS	eAPI over the private containerlab management network; management IPs validated with `ipaddress` before any shell; credentials via `curl -K -` on stdin	`executors/common.py::check_mgmt_ip`, `eapi_curl`
api/jobs → external services	https only for the update manifest; AI base URLs may not resolve to private/loopback/metadata addresses unless `ai_allow_private_urls`; SMTP AUTH only over STARTTLS/465 unless `AVDLAB_SMTP_ALLOW_INSECURE`; git `token` auth refused for `http://` and `git://`; local git remotes refused unless `AVDLAB_GIT_ALLOW_LOCAL`; tokens in headers, `GIT_ASKPASS` files or stdin, never on argv	`meta/update.py::fetch_latest`, `core/security.py::is_private_address`, `core/mailer.py`, `labs/git.py`
release → running instance	digest-pinned image required (`AVDLAB_UPDATE_REQUIRE_DIGEST`), optional keyless cosign verification (`AVDLAB_UPDATE_COSIGN_IDENTITY`, fail closed), admin-only endpoints	`meta/update.py::image_error`, `verify_image_signature`; `meta/api.py`

1.3 Authentication and session design

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks



Session life cycle

PROPERTY	DESIGN	EVIDENCE
password storage	argon2id (`argon2-cffi` defaults, RFC 9106 low-memory profile); recovery codes hashed with a cheaper argon2id profile	`core/passwords.py`
password policy	≥ 12 characters (`AVDLAB_PASSWORD_MIN_LENGTH` , floor 8), 3 of 4 character classes, not containing the username, not in the common-password list, ≤ 3 identical characters in a row	`core/passwords.py::password_violations`
brute force	per-IP token bucket 20/min on login and MFA verify (`AVDLAB_AUTH_RATE_LIMIT`); account lockout after 5 failures for 15 minutes; **uniform** `401 invalid username or password` for unknown user, wrong password and locked account (no enumeration); registration answers `202 pending` even for a taken username	`core/ratelimit.py` , `api/auth.py` , `api/registration.py`
MFA	TOTP (30 s steps) with a replay guard (`users.totp_last_step` : a step is accepted once), single-use recovery codes, 5 wrong codes restart the sign-in; TOTP seeds are Fernet-encrypted with the secret-store keys	`api/auth.py` , `core/db.py`
MFA policy	`mfa_required_roles` (default `["admin"]`); a listed role without an authenticator gets a **restricted session** that reaches only `/api/auth/*` and `/api/meta*` (`403 mfa_required` , WebSocket 4403)	`core/security.py::mfa_required_for` , `RoleGuard`
session token	server-side row in `sessions` (`user_id` , `created_at` , `last_seen_at` , `expires_at` , `ip` , `user_agent` , `revoked_at`); the cookie carries only `sid` , `uid` , `sv` and is signed by Starlette's `SessionMiddleware`	`core/security.py::create_session` , `session_user`
lifetimes	cookie 7 days sliding (`AVDLAB_SESSION_MAX_AGE`); absolute 12 h (`AVDLAB_SESSION_ABSOLUTE_MAX_AGE` = 43200 s, `session_lifetime()` interprets seconds) — re-authentication after 12 h regardless of activity	`core/config.py` , `core/security.py::session_lifetime`
revocation	sign-out revokes the row; the Account page lists and revokes sessions; `logout-all` ; admin *Disable*, *Reset password*, *Reset MFA* and *Delete* revoke every session of that user; a password change revokes the others	`api/auth.py` , `api/users.py`
signing key	`AVDLAB_SESSION_SECRET` empty → 64-hex secret generated once into `/data/session.key` (0600); an explicit value must be ≥ 32 characters and not the old default, else the process refuses to start	`core/security.py::resolve_session_secret`
bootstrap admin	created from `AVDLAB_ADMIN_USER` / `AVDLAB_ADMIN_PASSWORD` while the users table is empty; a weak password forces a change at first sign-in; with `AVDLAB_PUBLIC_URL` set a random 20-character password is generated and logged once	`api/users.py::bootstrap_admin`

There are **no API tokens or service accounts** in 1.1.0: every API call is a cookie session of a human user. Scripts sign in with `POST /api/auth/login` and keep the cookie jar (`docs/1.1/07-api.md` §7.5).

1.4 Authorisation model

Three roles, enforced twice: by the `RoleGuard` ASGI middleware (`core/security.py::role_denial` , a pure function with unit tests) on every `/api/*` request and WebSocket upgrade, and by `require_role("admin")` dependencies in the admin routers. Anonymous requests pass RoleGuard and are answered `401` by the routers.

ROLE	MAY	MAY NOT
`admin`	everything: users, account requests, admin settings, audit trail, update check/apply, every secret (metadata), every lab and host	—
`operator`	create and manage labs, hosts, images, targets, run every job, open web terminals, create and bind **own** secrets, change own password/MFA/sessions	`/api/users*` , `/api/admin*` , `/api/audit*` , `POST /api/meta/update/*` , `DELETE /api/hosts/*` , bind another user's secret, reserved secret names
`viewer`	`GET` / `HEAD` / `OPTIONS` , job-log and status WebSockets, `POST /api/auth/*` (own password, MFA, sessions)	any mutation, any `.../terminal` WebSocket (a shell is not a read)
anonymous	`/api/auth/login` , `/auth/mfa/verify` , `/auth/logout` , `/auth/policy` , `/api/register` , `/register/status/{id}` , `/api/meta` , `/meta/changelog` , `/meta/notices`	everything else (`401`)

Object-level scoping exists for **secrets** (`secrets.created_by` , `owns_secret` , `require_binding` with a compatible `purpose` ; legacy rows with `created_by` NULL are admin-only) and for **sessions** (a user sees and revokes only their own). **Labs, hosts, images and targets are shared** between all operators and admins: avdlab is a team lab tool, not a multi-tenant platform; separate tenants need separate deployments (risk register R-12).

Secret purposes and where they may be bound: `ssh` → host key, `host` → host password / sudo password, `git` → git remote auth, `arista` / `cloudvision` / `netbox` / `ai` → the respective target or link, `generic` → nowhere automatically. Reserved names `smtp_password` , `ai_default_api_key` , `github_token` and `system:*` are admin-only (`is_reserved_secret_name`). Note from `docs/_changes/A.md` : CloudVision and NetBox *target* token bindings (`cloudvision/api.py` , `netbox/api.py`) are not ownership-checked in 1.1.0 (risk register R-07).

1.5 Secret management

ASPECT	DESIGN
algorithm	Fernet (AES-128-CBC + HMAC-SHA256, `cryptography`), through `MultiFernet`
keys	`AVDLAB_SECRET_KEYS` comma-separated: **first encrypts, all decrypt**; `AVDLAB_SECRET_KEY` appended as decrypt-only when both are set; neither → key generated on first start into `/data/secret.key` (0600). Each key is validated at startup (`validate_fernet_key` : 32 bytes urlsafe-base64) and a malformed key stops the process
rotation	`python -m avdlab.cli rotate-secrets [--dry-run]` re-encrypts every `secrets.ciphertext` and `users.totp_secret` with the primary key; `check-keys` reports which key opens how many rows and exits 2 when a row opens with none. Procedure: `docs/1.1/08-operations.md` §8.6; playbook: chapter 07 §7.6
what is encrypted	every `secrets` row (SSH keys, passwords, arista/CloudVision/NetBox/AI tokens, SMTP password, GitHub token), every TOTP seed, lab `secret_knobs` (device password, `act.password` since 1.1.0)
what is not encrypted	user PII (username, e-mail, full name), lab content and build outputs (the device password is by design rendered into the AVD inputs — a lab credential, not a platform secret), job logs (redacted instead), the SQLite file itself

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

ASPECT	DESIGN
ownership	`created_by` (user id), `purpose`; operators see, delete and bind only their own rows; admins see everything with `created_by_name`; reserved names admin-only
exposure	list endpoints return names, kinds, `purpose`, `own`, never values; tokens on targets are write-only (`has_token`); only job and service code calls `secret_store.reveal()`, and every reveal is followed by `ctx.redact()`
audit	`secret.created`, `secret.deleted`, `secret.bound` with actor
key custody	the operator's: `/data/secret.key` and `/data/session.key` are on the data volume (root on the orchestrator host reads them); moving them to `.env` separates them from the database backup but puts them in a file the compose user reads. There is no HSM or KMS integration (risk register R-05)

Where each credential travels — the matrix of `docs/1.1/02-architecture.md` §(f), extended with the zone it reaches and the control at the far end:

CREDENTIAL	STORED	LEAVES ZONE 2 AS	REACHES	NEVER
lab host SSH private key / password	`secrets` (Fernet)	in memory to asyncssh	zone 5 sshd	disk on the orchestrator outside the store; argv; job log (`ctx.redact`)
sudo password	`secrets`	`sudo -S` on stdin	zone 5	command line
device password (eAPI)	lab `secret_knobs` **and** the AVD inputs (sha512/type-7) in the lab repo	`curl -K -` config on stdin; mgmt IP validated, URL `shlex.quote`d	zone 5 → cEOS	command line, log
ACT node password	`secret_knobs["act.password"]` (1.1.0)	ACT executor / build only; `********` in every lab response	ACT (zone 6)	plain text in `settings.knobs`; **exception:** the AI designer still receives it for ACT labs (§1.12, R-08)
ansible credentials (push, ACT deploy, test hosts)	derived in the job	stdin of `docker run -i`, `/tmp/avdlab-extra-vars.yml` 0600 inside the throw-away container	zone 5 agent container	host filesystem, `-e @/dev/stdin`
CloudVision service-account token	`secrets`	0600 env-file under `~/.avdlab/` on the lab host (`env_file_content` refuses control characters), `--env-file`, removed in `finally`	zone 5 → CloudVision	lab repo, command line
CloudVision onboarding token	`secrets`	stdin of `docker exec -i <node>` writing the TerminAttr token file; ZTP: `clab/ztp/www/` 0600 via stdin	cEOS → CloudVision	`docker cp`, lab repo
ANTA credentials (PSIRT)	derived	0600 env-file in `$HOME`, removed in `finally`	zone 5 ANTA container	same
arista.com token	`secrets`	env of the local `ardl` process, or stdin of the remote `ardl`	arista.com	logs
git remote token / SSH key	`secrets` (`purpose: git`)	throw-away `GIT_ASKPASS` reading a 0600 temp file / `GIT_SSH_COMMAND -i <0600 file>` with `StrictHostKeyChecking=accept-new` and `/data/git_known_hosts`; temp dir removed; `token` refused for `http://` / `git://`	GitHub or other remote	`.git/config`, stored URL, argv
NetBox token, CloudVision REST token	`secrets`	`Authorization` header (pynetbox / httpx), `verify_certs` per target	NetBox / CloudVision	logs (redacted)
AI API key	`secrets` (`ai_default_api_key` reserved, per-user `ai_links`)	request header; every provider error passes `mask()`; base URL SSRF-checked	AI provider	logs, HTTP responses, the prompt
SMTP password	`secrets` (`smtp_password` reserved) or env	aiosmtplib login over STARTTLS / 465 only (unless `AVDLAB_SMTP_ALLOW_INSECURE`)	SMTP server	logs, error messages, plaintext connection
session cookie	browser	`Cookie` header	zone 2	JavaScript (HttpOnly), cross-site POST (`SameSite=Lax` + OriginGuard)

1.6 External services: what each one sees

SERVICE	DATA SENT	CREDENTIAL	TRANSPORT
GitHub Releases API	nothing but the request (`User-Agent: avdlab-update-check`)	none	https (http manifests refused)
GitHub (feedback issues)	feedback title/body with diagnostics, secrets masked	`github_token` (admin-only secret)	https
git remotes (GitHub or other)	the whole lab repository including AVD inputs with the hashed device password	git token or SSH key (`purpose: git`)	https / ssh; `file://` only with `AVDLAB_GIT_ALLOW_LOCAL`
arista.com	download requests for cEOS images	arista.com token	https (eos-downloader)
CloudVision	device streaming (TerminAttr from the switches), `cv_deploy` structured configs and tags, inventory queries	service-account token, onboarding token	https; `verify_certs` per target (on-prem self-signed)
NetBox	sites, devices, interfaces, IPs, cables of the lab	API token	http(s) as configured (compose NetBox on the compose network)
AI provider	template descriptor, non-secret knob values, topology summary, lab name/state/target, the user's chat history; **the `act` knob including the ACT password for ACT labs** (`docs/1.1/06-ai-designer.md` §6.5)	API key in header	https; private/loopback base URLs refused unless `ai_allow_private_urls`

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

SERVICE	DATA SENT	CREDENTIAL	TRANSPORT
SMTP	account-request and feedback mails (usernames, e-mail addresses, reasons)	SMTP password	STARTTLS or implicit TLS
Let's Encrypt	domain validation (Caddy)	ACME account in `caddy_data`	https

1.7 Transport security

- **Public profile:** Caddy terminates TLS with Let's Encrypt certificates (`caddy_data` volume), redirects 80 → 443, sets `Strict-Transport-Security max-age=31536000` , removes `Server` , caps bodies (4 GB on `/api/images/upload` , 16 MB elsewhere) and proxies to `api:8080` on the compose network; the api port is bound to `127.0.0.1` . HTTP/3 is published on 443/udp. Caddy does **not** set a CSP — the API owns it.
- **LAN profile:** plain `http://host:8080` ; the cookie is not `Secure` unless `AVDLAB_SECURE_COOKIES=true` . This is acceptable only on a network the operator trusts end to end (checklist 09 §9.3).
- **Internal:** the api talks to Postgres, NetBox and the socket proxy over the compose network unencrypted; the compose network is local to the orchestrator host.
- **SSH:** asyncssh with host keys pinned on first use in `/data/known_hosts` ; mismatch fails the connection.
- **Outbound:** httpx with certificate verification for GitHub, AI providers and CloudVision (per-target `verify_certs` can be disabled for on-prem CloudVision with private CAs); eAPI inside the lab uses `curl -sk` (§1.12).

1.8 Container and host hardening

CONTROL	SETTING	FILE
non-root	`USER avdlab` (uid/gid 10001) in the image; `user: "10001:10001"` in compose; `/data` pre-chowned	`deploy/Dockerfile.backend` , `deploy/docker-compose.yml`
immutable root fs	`read_only: true` ; writable only `/data` (volume), `/tmp` and `/home/avdlab` (tmpfs, sized)	compose
no capabilities, no escalation	`cap_drop: [ALL]` , `security_opt: no-new-privileges:true`	compose
resource limits	`mem_limit` / `memswap_limit` 2g, `cpus` 2.0, `pids_limit` 512 (`AVDLAB_MEM_LIMIT` , `AVDLAB_CPUS` , `AVDLAB_PIDS_LIMIT`)	compose
no Docker socket	removed from the api; `docker-socket-proxy` (digest-pinned, read-only, `no-new-privileges` , socket `:ro` , no published port, allow-list of API sections) behind the `update` profile	compose
pinned supply	every `FROM` `tag@digest` ; `npm ci` only; `pip install --require-hashes --no-deps -r requirements.lock` ; third-party compose images digest-pinned (`netboxcommunity/netbox:${NETBOX_VERSION}` stays a tag because it is configurable)	Dockerfile, compose
no default secrets	`POSTGRES_PASSWORD` , `NETBOX_*` have no defaults; entrypoint wrappers exit when empty	compose
NetBox exposure	UI bound to `127.0.0.1` by default, `CORS_ORIGIN_ALLOW_ALL: "false"`	compose
health	`HEALTHCHECK` every 30 s against `/api/auth/me`	Dockerfile

The orchestrator **host** itself (OS patching, SSH hardening, firewall, disk encryption) is outside the product and on the hardening checklist (09).

1.9 Self-update supply chain

The release pipeline (chapter 05 §5.7) produces a multi-arch image on GHCR, a CycloneDX SBOM, SLSA provenance and a keyless cosign signature, and writes `<image>:<version>@sha256:<digest>` into the release notes. The consumer side, `meta/update.py` :

1. `AVDLAB_UPDATE_SOURCE` is `github:<owner>/<repo>` (Releases API) or an `https://` manifest URL; `http://` is refused (`fetch_latest`).
2. In image mode the release image must match `PINNED_IMAGE_RE` — `<image>:<tag>@sha256:<64 hex>` — taken from the manifest (`image` with `@sha256:` or a `digest` field) or from the release notes (`digest_from_notes`). Otherwise `GET /api/meta/update` reports `image_error` , `can_self_update` is false and the job refuses (`AVDLAB_UPDATE_REQUIRE_DIGEST` , default true).
3. With `AVDLAB_UPDATE_COSIGN_IDENTITY` set, `cosign verify --certificate-identity-regexp <identity> --certificate-oidc-issuer <issuer> <image>` runs **before** `deploy/.env` is touched; cosign missing → refused (fail closed, `verify_image_signature`). The identity to configure is `^https://github.com/<owner>/<repo>/.github/workflows/release.yml@refs/tags/v` .
4. The detached helper (`docker:27-cli@sha256:…` , digest-pinned) rewrites `AVDLAB_IMAGE=` in `deploy/.env` , runs `docker compose pull api && docker compose up -d api` , through the socket proxy (`DOCKER_HOST`), never with the socket mounted into the api.
5. `POST /api/meta/update/check|apply` are admin-only; `update.json` / `update.log` record `{from, to, mode, at, job_id, helper, ok}` and the next process closes the job (`finalize_previous_update`).

Source mode (`git pull --ff-only && docker compose up -d --build`) has no signature check — it trusts the git remote and needs `AVDLAB_PROXY_ALLOW_BUILD=1` ; public deployments should use image mode (checklist 09 §9.7).

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

1.10 Job isolation, redaction and limits

Jobs are asyncio tasks inside the api process (`jobs/engine.py`) — there is **no process or container isolation** between jobs, and a job runs with the api's privileges in zone 2. What limits the blast radius:

- **Redaction:** each `JobContext` has a `Redactor` ; `ctx.redact(*values)` registers revealed secrets (≥ 4 characters) and every later log line replaces them with `***` ; built-in patterns always apply (`Bearer …` , `token|password|passwd|pwd|secret|api_key` followed by `=` / `:` , `scheme://user:pass@host` , AWS `AKIA…` , PEM private-key blocks collapsed across lines). False positives are accepted (`has_secret=` values are masked too).
- **Caps:** `AVDLAB_JOB_LOG_MAX_BYTES` (20 MiB, log ends with `[log truncated]`), `AVDLAB_JOB_MAX_CONCURRENT` (4, FIFO queue), `AVDLAB_JOB_LOG_RETENTION_DAYS` (90, directories removed at startup and daily; rows stay).
- **Cancellation:** `POST /api/jobs/{id}/cancel` closes the SSH sessions the job holds so remote commands die with them; interrupted jobs are marked `failed` at the next start.
- **Input validation before shell:** mgmt IPs (`check_mgmt_ip`), env-file values (`env_file_content`), secret knobs (`check_secret_value` : no newline/CR/NUL), git branches (`check_branch`), jump hosts (`parse_jump`), `shlex.quote` on every interpolated argument.
- **Remote tree:** `/opt/avdlab` 0750, `labs/<slug>` 0700 shared with the agent image's gid, report subtrees `g+rwX` ; fallback to world-writable **report directories only** with a warning when the gid cannot be determined.

Lab **templates** are admin-controlled content: a template that echoes a secret in a shell step would defeat redaction (SECURITY.md → *By design*; risk register R-13).

1.11 Logging and audit design

Two streams. The **audit trail** (`audit_log` : `at` , `username` = target, `actor` = who, `event` , `ip` , `user_agent` , `detail`) records security-relevant events — sign-ins, lockouts, MFA, sessions, registration, users, secrets, hosts, labs, settings, `ai.apply` , `job.cancelled` — either explicitly (`audit.record`) or declaratively (`AuditMiddleware` on 2xx for `POST /labs` , `POST /labs/import` , `DELETE /labs/{id}` , `PUT /labs/{id}/files/{path}` , `DELETE /feedback/{id}` ; detail is `METHOD path` plus path parameters, never a body). Admins query it (`GET /api/audit` , filters, `limit` ≤ 1000) and export it (`GET /api/audit/export` , NDJSON, newest first, ≤ 100 000 rows); retention `AVDLAB_AUDIT_RETENTION_DAYS` (365; 0 = forever), pruned at startup and daily with dead sessions. The **operational logs** are the uvicorn/application stdout (collected by Docker) and the per-job `log.txt` files. The audit table is append-only by the application but not tamper-evident (no hash chain, no WORM): integrity depends on forwarding it off the host (chapter 06). Full inventory: chapter 06.

1.12 What is deliberately not protected

These are design decisions, stated so that a reviewer does not rediscover them as findings. Each has a row in the risk register (03) with its residual rating.

ITEM	WHY IT IS ACCEPTED	COMPENSATING FACTOR
Lab hosts are root-equivalent for avdlab and for every `operator` : `sudo containerlab` , `docker run` , bootstrap installs packages as root	that is the product's job; sandboxing an orchestrator from the hosts it orchestrates would make it useless	dedicated hosts/VMs, no shared credentials, sudo scope, host key pinning, audit of host changes (`host.created/updated/deleted`), secrets ownership (an operator can only bind their own key)
cEOS eAPI uses self-signed certificates; `curl -sk` and `anta … --insecure` skip verification	lab switches generate their own certificates; the management network is private to the lab host	mgmt IP validated with `ipaddress` ; credentials on stdin; the network is not routed
ACT node terminals use `StrictHostKeyChecking=no` + `UserKnownHostsFile=/dev/null` (`executors/terminal.py::act_node_command` , the interactive `ssh` from the ACT tools server to a node's mgmt IP)	ACT nodes are ephemeral cloud instances with changing keys; the password is typed interactively	containerlab node terminals use `docker exec` over the pinned lab-host SSH session and do not have this issue; ACT client sessions use `accept-new` with a per-lab `known_hosts`
The AI provider receives lab data (knobs, topology summary, chat) and, for ACT labs, the decrypted ACT password (`ai/api.py` → `labs.api._stored_knobs` → `designer.current_values` ; `act` is an `object` knob, not `secret`)	bring-your-own-model by design; the ACT leak is a known 1.1.0 gap, documented in `docs/1.1/06-ai-designer.md` §6.5	admin opt-in for a shared model, per-user links, local models with `ai_allow_private_urls` , device password and tokens never sent; mitigation: throw-away ACT password
The device password is in the lab repository (sha512 hash of the local user, type-7 BGP passwords, `ansible_password`)	it is an AVD input and must be there for the build to be reproducible	it is a lab credential, not a platform secret; git remotes are the operator's choice
Labs, hosts, images and targets are shared between operators	team tool, single tenant	separate deployments per tenant
Admins see everything (secret metadata, every lab, the audit trail) and can run `Reset password` / `Reset MFA`	an admin is the operator's trusted role	MFA required for admins by default, every admin action audited with `actor` , cannot demote/disable/delete the last admin or oneself
No tamper-evident log; SQLite is one file on one volume	single-node design	export/forwarding (06), backups (08), Postgres profile
Keys on the data volume by default	zero-configuration first start	`AVDLAB_SECRET_KEYS` / `AVDLAB_SESSION_SECRET` in `.env` , file modes 0600, non-root container
cosign verification is opt-in	the cosign CLI is not in the default image; forks may not sign	digest pinning is mandatory by default; fail closed when identity is set

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

CHAPTER 2

Data classification

Asset and data inventory: classes, storage, encryption, access, retention, backup.

This chapter lists what an avdlab 1.1.0 deployment stores and transmits, in six data classes, and states for each where it lives, how it is protected at rest and in transit, who can read it, how long it is kept and whether the backup procedure (chapter 08) covers it. The classes are the vocabulary used by the risk register (03), the logging chapter (06) and the incident playbook (07). Paths are inside the `avdlab-data` volume (`/data`) unless stated.

2.1 Classes

CLASS	SHORT NAME	DEFINITION	DEFAULT SENSITIVITY
C1	**Platform secrets**	anything that grants access to avdlab itself or to a system on its behalf: Fernet keys, session secret, SSH keys and passwords of lab hosts, API tokens (arista.com, CloudVision, NetBox, AI, GitHub), SMTP password, Postgres/NetBox passwords, TOTP seeds, recovery codes, password hashes	**Confidential — restricted**: compromise gives control of the platform or a third-party account
C2	**User PII**	account data of humans: username, e-mail, full name, registration reason, role, IP addresses and user agents in sessions and audit rows, AI chat history held in the browser	**Confidential — personal data** (GDPR applies to a deployment with EU users)
C3	**Lab credentials**	credentials that exist only inside a lab: the cEOS device password (`device_password` knob and its sha512 / type-7 renderings), ACT node password, CloudVision onboarding tokens written to switches, ZTP bootstrap scripts	**Internal**: scoped to a lab the operator can destroy; the device password is deliberately part of the lab repository
C4	**Lab content and configurations**	AVD inputs, structured and EOS configs, documentation, ANTA catalogs and reports, containerlab topologies, running-config snapshots, cEOS images, templates	**Internal**: reveals network design; cEOS images are Arista-licensed material
C5	**Telemetry and logs**	audit trail, job logs, application stdout, host monitor samples, update bookkeeping, feedback entries	**Internal**, with C1/C2 fragments possible (IPs, usernames; secrets masked in job logs)
C6	**AI prompts and responses**	what the designer sends to the configured model and what comes back: template descriptor, non-secret knob values, topology summary, user messages; **for ACT labs also the ACT password** (1.1.0 gap)	**Internal**, leaves the deployment to a third party; treat as C3 for ACT labs

2.2 Inventory

ASSET	CLASS	WHERE STORED	AT REST	IN TRANSIT	WHO CAN READ	RETENTION
Fernet key(s)	C1	`/data/secret.key` (0600) **or** `AVDLAB_SECRET_KEYS` / `AVDLAB_SECRET_KEY` in `deploy/.env`	file mode only; the volume is not encrypted by avdlab	never transmitted	root on the orchestrator host; uid 10001	until rotated (08 §8.6 of `docs/1.1`)
session signing secret	C1	`/data/session.key` (0600) or `AVDLAB_SESSION_SECRET`	file mode	never	same	until rotated
`secrets` rows (SSH keys, passwords, tokens, SMTP, GitHub, AI keys)	C1	`secrets.ciphertext` in `avdlab.db` / Postgres	Fernet (primary key encrypts, all decrypt)	revealed only inside jobs: stdin, 0600 files, HTTP headers (01 §1.5)	admins see metadata of all rows, operators their own; **values** only job/service code	until deleted (`DELETE /api/secrets/{id}`, audited)
TOTP seeds, recovery codes, password hashes	C1	`users.totp_secret` (Fernet), `users.recovery_codes` (argon2id hashes), `users.password_hash` (argon2id)	Fernet / argon2id	never	nobody through the API	account lifetime
lab `secret_knobs` (device password, `act.password`)	C3	`labs.settings.secret_knobs`	Fernet	stdin / env-file / ansible extra vars 0600 in a throw-away container	`********` in every lab response; job code	lab lifetime
Postgres / NetBox passwords, `NETBOX_SECRET_KEY`, `NETBOX_API_TOKEN`	C1	`deploy/.env`	file mode (operator's choice)	compose network	compose user, root	until rotated
user accounts (`users`, `account_requests`)	C2	database	plain (SQLite file / Postgres)	https (public) or LAN http	admins (`GET /api/users`, requests); the user (own `/auth/me`)	account lifetime; `account_requests` until approved/rejected and deleted b an admin

ASSET	CLASS	WHERE STORED	AT REST	IN TRANSIT	WHO CAN READ	RETENTION
sessions (`sessions` : ip, user_agent, timestamps)	C2	database	plain	—	the user (own), indirectly admins via audit	revoked rows pruned daily; expir rows a day after expiry
audit trail (`audit_log` : username, actor, event, ip, user_agent, detail)	C5 (+C2)	database	plain	NDJSON export over the API session	admins only (`/api/audit*`)	`AVDLAB_AUDIT_RETENTION_DAYS` (365; 0 = forever)
job logs	C5	`/data/jobs/<id>/log.txt` (+ artefacts)	plain, **secrets masked** `***` , capped 20 MiB	WebSocket / `GET /api/jobs/{id}/log`	any authenticated role (viewer included)	`AVDLAB_JOB_LOG_RETENTION_DA` (90) directories; DB rows stay
job rows (`jobs` : params, result, error)	C5	database	plain	API	any role	indefinite
application stdout (uvicorn, `avdlab.*` loggers)	C5	Docker json-file log on the orchestrator host (default driver)	plain	`docker logs`	root / docker group on the host	Docker driver default (unbounde unless configured)
Caddy logs	C5 (+C2 IPs)	container stdout; no access log unless a `log` directive is added to the Caddyfile	plain	—	root / docker group	Docker driver
host monitor samples (`host_metrics`)	C5	database	plain	—	any role	7 days (`hosts/monitor.py::RETENTION`
update bookkeeping	C5	`/data/update.json` , `update.log` , `update-check.json`	plain	—	admins (`GET /api/meta/update`)	overwritten per update
lab repositories	C4 (+C3)	`/data/labs/<slug>/` git repos: `inventory.yml` , `group_vars/` , `intended/` , `documentation/` , `anta/` , `clab/` , snapshots	plain	tar over SSH to `/opt/avdlab/labs/<slug>` (0700); git push to the configured remote	any role (viewer reads files)	lab lifetime (`DELETE /api/labs/{id}` , audited)
remote lab tree	C4 (+C3)	lab host `/opt/avdlab/labs/<slug>`	0700, shared with the agent gid	—	the SSH user, root, agent containers	until `lab.destroy` /re-sync
cEOS images	C4	`/data/images/<fmt>-lab-<version>.<ext>` ; copies on lab hosts	plain	SSH / `docker save`	any role	until deleted
templates	C4	image `/app/templates` (read-only)	—	—	all	per release
feedback entries	C5 (+C2)	`feedback` table, `/data/feedback/<id>.md` , optional GitHub issue, mail	plain; secrets masked in diagnostics	https to GitHub / SMTP TLS	any role reads; admin deletes (audited)	until deleted
AI prompts / responses	C6	not stored server-side; chat history in the browser (`localStorage`); AI link metadata in `ai_links`	provider's policy	https to the provider	the provider; the user	provider's retention
pending registration record	C2	browser `localStorage['avdlab.accountRequest']` = `{id, username, status}` only (1.1.0)	browser	—	the user's browser	removed once approved/rejecte
SSH host keys	C4	`/data/known_hosts` , `/data/git_known_hosts` , per-lab `act/clients/known_hosts` on the lab host	plain (public keys)	—	—	until removed by hand
TLS certificates / ACME account	C1	`caddy_data` volume	plain	—	root	renewed by Caddy

2.3 Encryption summary

LAYER	MECHANISM	COVERS	DOES NOT COVER
application-level at rest	Fernet / `MultiFernet` with rotatable keys	`secrets.ciphertext` , `users.totp_secret` , lab `secret_knobs`	the database file, PII, lab content, logs, the key files themselves
password storage	argon2id	user passwords, recovery codes	—
volume at rest	**none by avdlab**	—	everything: the operator provides disk encryption (LUKS, Azure disk encryption, FileVault on a Mac mini) when the threat model includes physical access or disk images (checklist 09)

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

LAYER	MECHANISM	COVERS	DOES NOT COVER
in transit, users	TLS 1.2+ by Caddy (Let's Encrypt), HSTS	the public deployment	the LAN deployment (`http://…:8080`)
in transit, lab hosts	SSH (asyncssh, pinned host keys)	commands, file sync, terminals, monitor	eAPI inside the lab (`curl -sk`, self-signed)
in transit, external	TLS (httpx/pynetbox/aiosmtplib; `verify_certs` per CloudVision/NetBox target; SMTP STARTTLS or 465)	GitHub, AI, CloudVision, NetBox, SMTP	a target with `verify_certs: false`; `AVDLAB_SMTP_ALLOW_INSECURE=true`
backups	**operator's responsibility**: `age` / `restic` / `gpg` before leaving the host (08)	the archive	—

2.4 Handling rules per class

C1 — Platform secrets. Never in the repository, a ticket, a chat or a job log. Enter them through the Secrets page or Admin settings (encrypted at rest) rather than `.env` where both are possible (`smtp_password`, `ai_default_api_key`). Keys in `.env` are read by the compose user: restrict `deploy/` to the operator account (`chmod 0700 deploy; chmod 0600 deploy/.env`). Rotate on suspicion and on offboarding of anyone who held them (07 §7.6–7.8). Back up key files with the database, encrypted, and store one copy outside the orchestrator host. Generated SSH keys belong to the user who created them; a shared "team key" is not allowed (04 §4.8).

C2 — User PII. Collect the minimum: the registration form asks for username, e-mail, full name and reason; the browser keeps only `{id, username, status}`. Access is admin-only. Delete accounts rather than disabling them indefinitely once the audit window has passed; the audit trail keeps usernames for `AVDLAB_AUDIT_RETENTION_DAYS`. A deployment with EU users has a controller (the operating organisation); record avdlab in its processing register with the categories above, the retention settings and the SMTP/AI processors it uses. Exports of the audit trail contain IPs and usernames: store them as C2.

C3 — Lab credentials. Assume they are visible to everyone with access to the lab repository, the lab host and any git remote the lab is pushed to. Use lab-only values (never a password that is also used elsewhere). The ACT password of an ACT lab is also visible to the configured AI model in 1.1.0: use a throw-away value or a local model. Destroying the lab and changing the knob is the rotation.

C4 — Lab content and configurations. Internal engineering data. cEOS images are Arista-licensed: keep `/data/images` and lab hosts within the licence terms and do not include images in off-site backups that others can read. Git remotes for labs are an outbound data flow the operator must approve (where do the configs go, who can read the remote).

C5 — Telemetry and logs. Job logs are readable by every role including `viewer`; redaction is strong but heuristic, so a template that prints a secret defeats it — review new templates (05 §5.11). Forward audit exports and container logs to a system with restricted access and its own retention (06). Treat exports as C2 because of IPs and usernames.

C6 — AI prompts. Decide per deployment which provider may see lab data; the admin form for the shared model says so explicitly. A provider's data-retention and training terms are part of the supplier review (05 §5.9). Local models behind `ai_allow_private_urls` keep C6 inside the network. Do not paste C1 values into the designer chat.

2.5 Data flows that leave the deployment

FLOW	TRIGGER	CLASS	CONTROL TO REVIEW
git push of a lab	operator configures a remote and pushes	C4 + C3	remote's access control; `token` only over https/ssh
AI design call	user with an AI link or shared model	C6 (+C3 for ACT)	provider terms; `ai_allow_private_urls` for local models
CloudVision onboarding / `cv_deploy`	operator binds a CV target	C4, C3 (onboarding token), telemetry of the switches	CVaaS tenant / on-prem instance ownership
NetBox sync	operator binds a NetBox target	C4	NetBox instance ownership
feedback → GitHub issue / mail	user submits feedback; admin configured repo/mail	C5 (+C2)	repository visibility, mail recipients
account-request mail	public registration	C2	`AVDLAB_REGISTRATION_MAIL_TO`, SMTP TLS
audit export	admin downloads NDJSON	C5 + C2	where the file is stored
backup archive	operator's cron	**all classes including C1**	encryption before leaving the host (08)
update check	daily `GET` to GitHub	none (version strings)	—

2.6 Deletion and retention controls

What the product deletes on its own, what an admin deletes through the API, and what needs a shell on the orchestrator. "Hard" means the bytes are gone from the live store (backups are a separate copy, 08 §8.3).

DATA	AUTOMATIC	THROUGH THE API / UI	ONLY BY HAND	NOTES
audit rows	pruned when older than `AVDLAB_AUDIT_RETENTION_DAYS` (startup + daily)	— (admins cannot delete rows)	`sqlite3 … DELETE FROM audit_log`	by design no API deletes audit rows; export before they age out
sessions	revoked rows pruned daily; expired rows a day after expiry	revoke / logout-all / disable	—	

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

DATA	AUTOMATIC	THROUGH THE API / UI	ONLY BY HAND	NOTES
job directories (`log.txt` , artefacts)	removed after `AVDLAB_JOB_LOG_RETENTION_DAYS`	—	`rm -rf /data/jobs/<id>` (07 §7.10)	job rows stay indefinitely
host monitor samples	7 days	—	—	
secrets	—	`DELETE /api/secrets/{id}` (owner or admin), audited	—	hard; a bound host keeps a dangling id until re-pointed
users	—	`DELETE /api/users/{id}` , audited	—	hard; audit rows keep the username (pseudonymous after deletion)
account requests	—	approve / reject (rows kept with the decision)	`DELETE FROM account_requests`	contains e-mail, name, reason — purge rejected requests periodically
labs	—	`DELETE /api/labs/{id}` removes the row and the repository directory, audited	—	lab trees on the lab host are removed by `lab.destroy` ; the git remote keeps its copy
cEOS images	—	Images page delete	`rm /data/images/…`	licensed material
feedback	—	admin delete, audited	—	GitHub issue / mail copies remain where sent
AI link	—	`DELETE /api/ai/link` (`ai.link.removed`)	—	the API key row is deleted
backups	restic `forget` policy (08 §8.4)	—	—	old archives decrypt with old Fernet keys

A **data-subject request** (access or erasure, GDPR art. 15/17) for a user is served with: `GET /api/users` (the account), `GET /api/audit?username=<u>` and `?actor=<u>` (events about and by them), the `sessions` rows (visible to the user on the Account page), feedback they filed, and the account-request row; erasure = `DELETE /api/users/{id}` plus purging their account request and, if required, replacing their username in `audit_log` rows by hand (the trail then loses attribution — record the legal basis for keeping it instead, usually legitimate interest in security logging for the retention period).

2.7 Data location

STORE	WHERE THE BYTES PHYSICALLY ARE	CONTROLLED BY
`avdlab-data` , `avdlab-pg` , `caddy_data` , NetBox volumes	the orchestrator host's Docker data root (`/var/lib/docker/volumes/…`), i.e. the Mac mini's disk or the Azure VM's managed disk	operator
remote lab trees, cEOS containers, agent containers	each lab host's disk (`/opt/avdlab` , Docker data root)	operator (per host)
backups	wherever 08 §8.4 sends them	operator; storage provider
CloudVision data	CVaaS region chosen by the tenant, or the on-prem cluster	Arista / operator
NetBox data	the compose volumes (profile `netbox`) or the external NetBox	operator
AI prompts	the provider's infrastructure, or the local model host with `ai_allow_private_urls`	provider / operator
GitHub (lab remotes, feedback issues)	GitHub's infrastructure	GitHub
mail	the SMTP provider and recipients	provider

For a deployment subject to data-residency requirements, the first two rows are under the operator's control; the remaining ones are supplier choices to record in the processing register (05 §5.9).

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

CHAPTER 3

Risk register

STRIDE-style risks with existing controls, residual risk and owners.

A STRIDE-style register over the trust-zone diagram of chapter 01 for avdlab 1.1.0. It is a **starting register for the operating organisation**: the likelihood and impact ratings assume a typical deployment (one orchestrator, a handful of operators, lab hosts in a lab network, optionally public HTTPS) and must be re-rated for the organisation's own context. "Existing control" names the file or setting that implements it; "residual" is what remains after that control with the shipped defaults; "owner" is who can lower it further. Review dates are placeholders.

3.1 Rating scale

LIKELIHOOD	MEANING	IMPACT	MEANING
L1 rare	needs a privileged insider or a chain of failures	I1 minor	one user, one lab, recoverable in minutes
L2 unlikely	needs a non-default misconfiguration or a 0-day	I2 moderate	several labs or a lab host; hours of recovery; internal data
L3 possible	plausible with public exposure and ordinary tooling	I3 major	platform secrets or a third-party account exposed; all users re-enrol
L4 likely	expected over the lifetime of a public deployment	I4 severe	persistent root on the orchestrator or lab hosts; personal data breach with notification duty

Residual = likelihood × impact after the existing controls: **Low** (≤ L2×I2), **Medium**, **High** (≥ L3×I3 or any I4 with L ≥ L2).

3.2 Register

ID	STRIDE	THREAT	ASSET (CLASS)	L	I	EXISTING CONTROL (FILE / SETTING)	RESIDUAL
R-01	Spoofing	credential stuffing / brute force against `POST /api/auth/login` on a public deployment	accounts (C2), everything behind them	L3	I3	per-IP token bucket 20/min (`core/ratelimit.py`), lockout 5 failures / 15 min, argon2id, uniform `401` (`api/auth.py`), MFA required for admins (`mfa_required_roles`), password policy (`core/passwords.py`)	**Low** — a distributed attacker bypasses the per-IP bucket; MFA and the lockout remain
R-02	Spoofing	session cookie theft or replay (XSS, malware on the endpoint, shoulder-surfed cookie)	session (C1)	L2	I3	HttpOnly, `SameSite=Lax`, `Secure` behind https; server-side `sessions` row (revocable, 12 h absolute, `session_version`); CSP `script-src 'self'`; `safeHref()` for every data-derived link; OriginGuard	**Low** — a stolen cookie works until revoked or 12 h
R-03	Spoofing	forged session cookie through a weak or leaked `AVDLAB_SESSION_SECRET`	all sessions (C1)	L1	I4	old default and < 32 chars refused at startup (`resolve_session_secret`); auto-generated 64-hex into `/data/session.key` 0600; `.env.example` ships it empty	**Low**
R-04	Tampering	cross-site request forgery against the cookie session	labs, hosts, secrets (C1/C4)	L2	I2	`SameSite=Lax`; `OriginGuard` on every mutating `/api` call and WebSocket upgrade (`403 origin not allowed` / 4403); CORS list limited to `AVDLAB_CORS_ORIGINS` + `AVDLAB_PUBLIC_URL`	**Low**
R-05	Information disclosure	Fernet key and database on the same volume: root on the orchestrator host, a stolen disk image or an unencrypted backup yields every secret and TOTP seed	platform secrets (C1)	L2	I4	key file 0600, container uid 10001 with read-only root fs; `AVDLAB_SECRET_KEYS` can move keys to `.env`; rotation tooling (`avdlab.cli`)	**High** — no HSM/KMS, no volume encryption by the product; a single-host design
R-06	Information disclosure	secrets leak into job logs (any role reads them) through a command that echoes them or a pattern the redactor misses	C1 in C5	L2	I3	`ctx.redact()` at every reveal point, built-in patterns (Bearer, key=value, URL creds, PEM, AKIA), stdin/0600 transport never argv, `redact_text` tests (`tests/test_platform_11.py`)	**Medium** — heuristic; templates are admin content and can print anything
R-07	Elevation of privilege	an `operator` uses another user's credential: secret bindings not ownership-checked for CloudVision / NetBox target tokens (`cloudvision/api.py`, `netbox/api.py`, noted in `docs/_changes/A.md`); legacy NULL-owner rows usable by admins only	tokens (C1)	L2	I2	ownership + purpose enforced on host and git bindings (`require_binding`); operators list only own secrets, so another user's id must be guessed/known; `secret.bound` audit	**Medium**

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

ID	STRIDE	THREAT	ASSET (CLASS)	L	I	EXISTING CONTROL (FILE / SETTING)	RESIDUAL
R-08	Information disclosure	the AI designer could see secret material inside object knobs (the ACT node password sits in `knobs.act`, an `object` knob, not a `secret` knob)	ACT password (C3) in C6	L1	I2	closed in 1.1.0: `ai/api.py` applies `mask_act_knob` before `designer.current_values`, so the model receives `********`; `secret` knobs were never sent	**Low**
R-09	Elevation of privilege	an `operator` registers or re-points a lab host and runs commands as root on it — or on a machine that is not a lab host	lab hosts (zone 5)	L2	I3	**by design** (SECURITY.md); own secrets only; `host.created/updated` audited with actor; TOFU host keys pinned; `DELETE /api/hosts` admin-only	**Medium** — the product trusts operators with their hosts
R-10	Tampering	swapped or impersonated lab host (MITM on the SSH path) steals the SSH credential or receives the lab	host credentials (C1), lab content (C4)	L1	I3	host key pinned on first use in `/data/known_hosts` (jump hosts too); mismatch fails; `git_known_hosts` for remotes	**Low** — first contact is unauthenticated (TOFU)
R-11	Tampering	malicious or tampered release applied through the self-update (compromised GitHub account, registry, or manifest host)	the api image (zone 2)	L1	I4	`https://` manifests only; digest-pinned image mandatory (`PINNED_IMAGE_RE`, `AVDLAB_UPDATE_REQUIRE_DIGEST`); optional keyless cosign with fail-closed (`verify_image_signature`); admin-only apply; helper image digest-pinned; release built in CI from an immutable tag with SLSA provenance	**Medium with cosign off (default), Low with cosign on** — a digest only proves what was pulled is what the notes said; the signature proves who built it
R-12	Information disclosure / Tampering	single tenancy: every operator sees and may change every lab, host, image and target	C3, C4	L3	I2	by design; audit of lab/host changes; secrets scoped per user	**Medium** (accepted)
R-13	Tampering	a lab template or a lab file edited through `PUT /labs/{id}/files/{path}` injects shell content executed on a lab host	lab host (zone 5)	L2	I3	templates ship in the image (admin content); file writes audited (`lab.file_written`); arguments `shlex.quote`d; mgmt IPs and env values validated; the lab host is already root-equivalent for operators (R-09)	**Medium** (collapses into R-09)
R-14	Denial of service	resource exhaustion of the api: large uploads, many jobs, log floods, memory	availability	L3	I2	`AVDLAB_UPLOAD_MAX_BYTES` 4 GiB + Caddy caps (4 GB / 16 MB); `AVDLAB_JOB_MAX_CONCURRENT` 4; log cap 20 MiB; compose `mem_limit` / `cpus` / `pids_limit`; `tmpfs` sizes; rate limits on login, registration (5/h/IP), AI (30/h/user)	**Low**
R-15	Elevation of privilege	container escape or Docker-API abuse from the api container reaches the orchestrator host	orchestrator host (zones 3–4)	L1	I4	no socket mount; `docker-socket-proxy` allow-list (no EXEC/AUTH/SECRETS/SWARM/SYSTEM; BUILD only with `AVDLAB_PROXY_ALLOW_BUILD=1`), socket `:ro`, no published port; api uid 10001, `cap_drop ALL`, `no-new-privileges`, read-only fs	**Medium** — the proxy still permits `POST /containers/create` with bind mounts, i.e. root-equivalent on the host when the `update` profile is on; source-mode updates need BUILD
R-16	Information disclosure / Spoofing	compromised third-party dependency or GitHub Action (supply chain into the build)	the image, the pipeline	L2	I4	hash-pinned `requirements.lock`, `package-lock.json` with `npm ci`, every `FROM` and action pinned by digest/SHA, Dependabot weekly, `pip-audit` / `npm audit` /Trivy gates, CodeQL, minimal workflow permissions, `persist-credentials: false`	**Low–Medium** — pins prevent silent drift, not a malicious upstream release that is then pinned
R-17	Repudiation	an admin or operator denies an action; the audit trail is modified or lost (SQLite file, no hash chain)	audit trail (C5)	L2	I2	`actor` on every row, IP and user agent, `AuditMiddleware` for lab routes, admin-only read, 365-day retention, NDJSON export	**Medium** — not tamper-evident; root on the host can edit the file
R-18	Information disclosure	user PII (e-mail, name, IPs) exposed through a database copy, an audit export left on a laptop, or mail to a wrong recipient	C2	L2	I3 (notification duty)	admin-only endpoints; browser keeps only `{id, username, status}`; SMTP TLS enforced; exports are on-demand	**Medium**
R-19	Spoofing	SSRF through operator-supplied URLs (AI base URL, git remote, CloudVision/NetBox server, update manifest) reaches internal services or the cloud metadata endpoint	internal network, cloud credentials	L2	I3	AI base URLs resolving to loopback/link-local/RFC 1918/CGNAT/metadata refused unless `ai_allow_private_urls` (`is_private_address`); `http://` manifests refused; local git remotes refused unless `AVDLAB_GIT_ALLOW_LOCAL`; `-`-prefixed URLs refused	**Medium** — CloudVision/NetBox server fields are not private-address-checked (by design: on-prem instances live on private addresses)
R-20	Denial of service / Information disclosure	loss of the only admin (forgotten password, lost authenticator, left the organisation) in a small deployment	availability of administration	L3	I2	`Reset password` / `Reset MFA` by another admin; recovery codes; last-admin guards; documented recovery (`sqlite3 … DELETE FROM users` re-bootstraps from env — destroys all accounts)	**Medium** — single-admin deployments have only the destructive path

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

ID	STRIDE	THREAT	ASSET (CLASS)	L	I	EXISTING CONTROL (FILE / SETTING)	RESIDUAL
R-21	Tampering	ACT node web terminal: `StrictHostKeyChecking=no` + `/dev/null` for the interactive `ssh` from the ACT tools server to a node (`executors/terminal.py::act_node_command`) allows a MITM inside the ACT environment to capture the typed password	ACT password (C3)	L1	I1	ACT environment is Arista-hosted and ephemeral; containerlab node terminals use `docker exec` over the pinned lab-host SSH; ACT client sessions use `accept-new` with a per-lab `known_hosts`	**Low** (accepted)
R-22	Information disclosure	eAPI to cEOS without certificate verification (`curl -sk`, `anta psirt --insecure`)	device password (C3)	L1	I1	lab management network private to the lab host; mgmt IPs validated; credentials on stdin	**Low** (accepted by design)
R-23	Denial of service	key or backup loss: `/data/secret.key` deleted, backup unreadable, keys rotated without re-encrypting	all secrets and MFA enrolments	L2	I3	`check-keys` exit 2 when rows are unreadable; rotation keeps old keys until verified; backup chapter 08	**Medium** — depends entirely on the operator's backup discipline
R-24	Information disclosure	LAN deployment over plain http: cookie and credentials readable on the network	sessions, passwords (C1)	L3 (on an untrusted LAN)	I3	`Secure` cookie derived from `AVDLAB_PUBLIC_URL`; the `public` profile with Caddy; checklist guidance	**High on an untrusted LAN, Low on a trusted one**

3.3 Known limitations at 1.1.0 (summary)

Stated once, honestly, so a reviewer can quote them:

1. **No volume encryption, no HSM/KMS** (R-05): the Fernet and session keys live on the data volume by default.
2. **Audit trail is not tamper-evident** (R-17) and **SQLite is a single file** on one host; forward exports and back up.
3. **CloudVision / NetBox target token bindings are not ownership-checked** (R-07).
4. **cosign verification of self-updates is opt-in** (R-11) and requires the cosign CLI in the image.
5. **ACT node terminals do not pin host keys** (R-21).
6. **Lab hosts are root-equivalent** for every `operator` (R-09) — by design.
7. **Single tenancy** (R-12) — by design.
8. **Small deployments with one admin** have only a destructive recovery path (R-20).
9. **The socket proxy, when started, still allows container creation** (R-15); it removes the socket from the api, it does not make Docker access safe.
10. **No API tokens / service accounts**: automation uses a human's cookie session (04 §4.6).
11. `ruff format` **backlog and dev-only npm advisories** (05 §5.3, §5.4) — no runtime exposure, tracked.

3.4 Review procedure

- Re-rate the register at every minor release and after every incident (07 §7.12). Record the date in the *Review* column and keep the previous version in git history.
- Add a row for every new external integration, new shell seam on a lab host, or new role capability.
- A row whose residual is **High** needs a decision recorded in the organisation's risk treatment plan: accept (with a named owner), mitigate (with a deadline), or avoid (do not deploy that way).
- The follow-ups owned by the maintainers (R-07, R-21) are tracked as issues labelled `security`; closing one updates this register and `docs/1.1`.

3.5 Coverage by trust zone and STRIDE

A quick check that every boundary of the trust-zone diagram (01 §1.1) has at least one row, and which STRIDE categories dominate it.

ZONE / BOUNDARY	ROWS	S	T	R	I	D	E
0 → 2 browser to api	R-01, R-02, R-03, R-04, R-14, R-24	●	●		●	●	
2 application	R-06, R-07, R-13, R-19	●	●		●		●
2 → 3 data volume / keys	R-05, R-17, R-18, R-23		●	●	●	●	
2 → 4 Docker control plane	R-15						●
2 → 5 lab hosts	R-09, R-10, R-21, R-22	●	●		●		●
2 → 6 external services	R-08, R-19	●			●		
release → 2 supply chain	R-11, R-16	●	●				
organisation	R-12, R-20		●		●	●	

Repudiation has a single row (R-17) because the product's answer to it is uniform: `actor` on every audit row and an export the admins cannot alter. Elevation of privilege inside the application is covered by the role model (04) and appears here only where the model is deliberately permissive (R-09, R-13, R-15).

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

3.6 Treatment plan template

For every row rated **High** or **Medium** the operating organisation records a decision. Suggested starting decisions for the shipped defaults; replace with your own.

ID	DECISION	MEASURE	OWNER	DUE	STATUS
R-05	mitigate	disk encryption on the orchestrator; keys in `.env` with 0600 and separate backup custody (08 §8.3); quarterly `check-keys`	operator		
R-24	avoid / mitigate	public profile (Caddy TLS) even on a LAN, or VPN-only access; never plain http over Wi-Fi	operator		
R-11	mitigate	`AVDLAB_UPDATE_COSIGN_IDENTITY` set; cosign CLI in the image; image mode	operator		
R-15	mitigate	`update` profile started only for an update window; `AVDLAB_PROXY_ALLOW_BUILD=0`; image mode	operator		
R-17	mitigate	weekly audit export to WORM storage (06 §6.4)	operator		
R-06	accept + monitor	template review on change; 90-day retention; purge procedure (07 §7.10)	operator		
R-07, R-08, R-21	mitigate (product)	tracked issues for the maintainers; interim: throw-away ACT passwords, CV/NetBox targets created by admins only	maintainers / operator		
R-09, R-12	accept	by design; dedicated lab hosts, one deployment per team	operator		
R-19	mitigate	egress filtering from the orchestrator to the services actually used	operator		
R-20	mitigate	second admin; sealed recovery codes with the backup keys	operator		
R-23	mitigate	08 §8.4 schedule, §8.7 quarterly restore test	operator		

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

CHAPTER 4

Access control policy

Roles, account lifecycle, MFA, passwords, sessions, access reviews, offboarding.

A policy template for an organisation operating avdlab 1.1.0. The technical statements (what a role can do, which settings exist, what the UI offers) are facts of the product; the procedural statements (who approves, how often to review) are **recommended defaults** the organisation adopts or adapts. Where the product enforces a rule the file is named; where only a procedure can, the step is written as a checklist.

4.1 Roles

Three application roles, enforced by `core/security.py::role_denial` (middleware) and `require_role("admin")` (routers). There is no custom role and no per-object permission beyond secrets and sessions.

CAPABILITY	ADMIN	OPERATOR	VIEWER
sign in, change own password, enrol/disable own MFA, list and revoke own sessions (`/api/auth/*`)	yes	yes	yes
read labs, hosts, images, targets, jobs, job logs (REST + log WebSockets)	yes	yes	yes
create / edit / delete labs, run every lab job (`build` , `deploy` , `validate` , `destroy` , push, PSIRT, traffic, snapshots)	yes	yes	no
create / edit hosts, bootstrap, probe, images (download, upload, import)	yes	yes	no
delete a host (`DELETE /api/hosts/{id}`)	yes	**no**	no
open a web terminal (`.../terminal` WebSocket) — a shell on a lab host or a switch	yes	yes	**no**
create secrets, bind **own** secrets to hosts / git	yes	yes (own, compatible `purpose`)	no
see all secrets' metadata (`created_by` , `created_by_name`), bind any secret, use legacy rows, reserved names (`smtp_password` , `ai_default_api_key` , `github_token` , `system:*`)	yes	no	no
CloudVision / NetBox targets, AI link of own account	yes	yes	read only
user administration (`/api/users*`): create, role, disable, delete, reset password, reset MFA, unlock; account requests approve/reject	yes	no	no
admin settings (`/api/admin/settings`), including `mfa_required_roles` , `ai_allow_private_urls` , AI shared model, mail	yes	no	no
revoke another user's sessions (`POST /api/users/{id}/sessions/revoke` , Users page *Sign out*)	yes	no	no
audit trail (`/api/audit` , `/api/audit/export` , page *Admin › Audit log*)	yes	no	no
update check / apply (`POST /api/meta/update/check|apply`), build facts (`GET /api/meta/build` : any authenticated user)	yes	no (build facts yes)	no (build facts yes)
feedback: submit (`POST /api/feedback`), delete (admin)	yes	yes	**no** (a mutation; viewers may only `GET` and `POST /api/auth/*`)

Denied HTTP calls answer `403 {"detail": "role viewer may not POST /api/labs"}` ; denied WebSockets close with `4403` . A restricted session (role requires MFA, none enrolled) answers `403 mfa_required` outside `/api/auth/*` and `/api/meta*` .

Assignment rule. `admin` is for the people who run the deployment (two minimum, see §4.4); `operator` for engineers who build and run labs; `viewer` for observers (students, reviewers, management). Because every `operator` is root-equivalent on every registered lab host (01 §1.12), granting `operator` is equivalent to granting root on those hosts: approve it accordingly.

4.2 Account lifecycle

```
request (public form)  →  approval (admin)  →  active (role)  →  changes (role / password / MFA)  →  disabled  →  deleted
            or
create (admin, temporary password)  →  first sign-in (forced change)  →  active …
```

STEP	PRODUCT BEHAVIOUR	PROCEDURE
Request	`POST /api/register` (when `registration_enabled`): username, e-mail, full name, reason → `202 pending` , 5 requests/hour/IP, audited `registration.requested` / `registration.duplicate` ; mail to `AVDLAB_REGISTRATION_MAIL_TO` when SMTP is set	The requester is identified out of band (the e-mail domain alone is not identity). Disable public registration (`registration_enabled=false`) when every user is created by an admin.
Approval	Users page → *Requests* → approve with a role (`POST /api/users/requests/{id}/approve`) or reject; audited `registration.approved` / `rejected` ; the approver's name is `actor`	Approve with the least role that does the job; `admin` approvals need a second admin's agreement recorded (ticket or chat), because the product has no four-eyes workflow.

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

STEP	PRODUCT BEHAVIOUR	PROCEDURE
Direct creation	`POST /api/users` with a temporary password (`must_change_password`), audited `user.created`	Hand the temporary password over a channel different from the username (phone vs mail).
First sign-in	forced password change (`must_change_password`); admins without an authenticator land in a restricted session until MFA is enrolled	—
Role change	`PATCH /api/users/{id}` `{role}` ; audited `user.updated` with `role a→b` ; cannot demote yourself or the last admin	Ticketed and approved by an admin other than the subject.
Disable	`PATCH /api/users/{id}` `{disabled: true}` ; **all sessions revoked immediately** (`revoke_user_sessions`); the row and its secrets remain; cannot disable yourself or the last admin	Default action on leave of absence, suspected compromise, or offboarding day 0.
Delete	`DELETE /api/users/{id}` ; sessions revoked, row removed; audited `user.deleted` . Secrets with `created_by` = the deleted user keep that id (they become unowned rows visible to admins only)	After the offboarding checklist (§4.9) and once nothing of the user's is still bound.
Unlock	`POST /api/users/{id}/unlock` after a lockout (the user sees only a generic 401)	Verify identity before unlocking; look at `login.failed` rows for the IP.

4.3 Authentication policy

RULE	PRODUCT ENFORCEMENT	SETTING
passwords ≥ 12 characters, 3 of 4 classes, not the username, not a common password, ≤ 3 repeated characters	`core/passwords.py::password_violations` on every set/change	`AVDLAB_PASSWORD_MIN_LENGTH` (12, floor 8)
no forced periodic rotation; change on suspicion, on reset, on offboarding	`must_change_password` after admin reset or bootstrap	—
5 consecutive failures → 15-minute lockout, invisible to the attacker	`api/auth.py` , uniform `401`	`AVDLAB_LOGIN_MAX_FAILURES` , `AVDLAB_LOGIN_LOCKOUT_MINUTES`
20 attempts/min per IP on login and MFA verify	`core/ratelimit.py`	`AVDLAB_AUTH_RATE_LIMIT` (0 disables — do not)
bootstrap admin password must be strong; behind a public URL a random one is generated and logged once	`api/users.py::bootstrap_admin`	`AVDLAB_ADMIN_PASSWORD` , `AVDLAB_PUBLIC_URL`
password hashes argon2id; TOTP seeds Fernet-encrypted; recovery codes hashed	`core/passwords.py` , `core/db.py`	—

4.4 Multi-factor authentication

- **Required for** `admin` by default: `mfa_required_roles = ["admin"]` (admin setting; env default `AVDLAB_MFA_REQUIRED_ROLES`). An admin without an authenticator can only reach `/api/auth/*` and `/api/me*` until enrolled (`core/security.py::mfa_required_for` , `mfa_restricted_path_allowed`).
- **Recommended:** add `operator` on any deployment reachable from outside the lab network (`PUT /api/admin/settings {"mfa_required_roles": ["admin", "operator"]}`), because operators hold root on lab hosts.
- TOTP: 6 digits, 30 s steps, **a step is accepted once** (`users.totp_last_step`); 5 wrong codes restart the sign-in. Recovery codes: single use, regenerate after using one (`POST /api/auth/mfa/recovery-codes`).
- Admin `Reset MFA` (`POST /api/users/{id}/mfa/reset`) clears the seed, bumps `session_version` , revokes all sessions and is audited `mfa.reset` ; the user must re-enrol at next sign-in. Verify identity out of band before resetting.
- **Two admins minimum.** With one admin, a lost authenticator and lost recovery codes leave only the destructive recovery (`DELETE FROM users` + restart re-bootstraps from env and deletes every account). Keep a sealed copy of one admin's recovery codes with the backup keys (08).

4.5 Session policy

RULE	VALUE	WHERE
idle lifetime (sliding cookie)	7 days	`AVDLAB_SESSION_MAX_AGE` = 604800
absolute lifetime (server-side row)	**12 hours** — re-authentication regardless of activity	`AVDLAB_SESSION_ABSOLUTE_MAX_AGE` = 43200 s (`session_lifetime()` in seconds)
cookie flags	HttpOnly always; `SameSite=Lax` ; `Secure` when `AVDLAB_PUBLIC_URL` is https or `AVDLAB_SECURE_COOKIES=true`	`main.py`
concurrent sessions	allowed; each is a row with IP and user agent; the user sees them on the Account page	`GET /api/auth/sessions`
user-initiated revocation	sign out (current), revoke one (`POST /api/auth/sessions/{sid}/revoke`), sign out everywhere (`POST /api/auth/logout-all`)	Account page
administrative revocation	there is **no admin endpoint that revokes another user's sessions directly**; *Disable*, *Reset password*, *Reset MFA* and *Delete* each revoke all of that user's sessions. To force a sign-out without changing anything else: disable, then re-enable	`api/users.py`
password change	revokes every other session of the user	`POST /api/auth/password`
secret rotation	changing `AVDLAB_SESSION_SECRET` (or deleting `/data/session.key`) and restarting invalidates every cookie at once	07 §7.5

RULE	VALUE	WHERE
shared workstations	users sign out; the 12 h absolute cap bounds a forgotten session	—

Recommended for public deployments with sensitive labs: shorten the absolute lifetime to a working day (`AVDLAB_SESSION_ABSOLUTE_MAX_AGE=28800`).

4.6 Automation and API access

1.1.0 has **no API tokens, no service accounts and no OAuth**. A script authenticates like a user (`POST /api/auth/login`, cookie jar, optional MFA step) and inherits that user's role and session lifetime (`docs/1.1/07-api.md` §7.5). Policy:

- Create a dedicated user per automation purpose (`operator` or `viewer`, never `admin`), owned by a named person, with MFA disabled only if its role is not in `mfa_required_roles` (an automation user cannot complete TOTP).
- Its password is a C1 secret (02 §2.4), rotated when the owner changes.
- Review automation users in the quarterly access review (§4.8) like any other account; the `sessions` list and `login.ok` rows show from where they sign in.

4.7 Least privilege on lab hosts

What avdlab needs on a lab host (`hosts/recipes.py`, `docs/1.1/04-lab-hosts.md`):

NEED	WHY	MINIMUM
SSH login as a dedicated user	all commands, file sync, terminals, monitor	key-based (generated on the Hosts page, purpose `ssh`); passwords only for the beta Mac/Windows hosts
`sudo`	`containerlab deploy/destroy/inspect`, bootstrap steps (packages, Docker repo, `/opt/avdlab` 0750), `prepare_agent_dir` (`chgrp` / `chmod` from a root container)	`NOPASSWD` is convenient but not required: avdlab sends the sudo password with `sudo -S` on stdin when a sudo secret is bound. A sudoers rule limited to `/usr/bin/containerlab`, `/usr/bin/docker`, `apt-get` / `dnf`, `install`, `mkdir`, `chown`, `chmod`, `usermod` covers the recipe; test with a bootstrap run
`docker` group membership	`docker run` of agent, ANTA, client containers; `docker exec` terminals; image import	bootstrap step *docker group membership* (`usermod -aG docker <user>`). Note that the `docker` group is itself root-equivalent on that host
outbound https	`download.docker.com`, `github.com/srl-labs/containerlab/releases`, `ghcr.io` (agent, ANTA, clab, client images), optionally CloudVision and arista.com	egress allow-list on the lab network
inbound	SSH from the orchestrator only (and the jump host's second hop)	firewall / NSG

Rules: one dedicated VM or machine per lab host; no personal accounts; no credential reused from another system; the lab management network (`clab` bridge, cEOS mgmt) not routed beyond the host; host keys verified on first probe (the fingerprint appears as `[tofu] pinned …` in the probe job log — compare it with `ssh-keyscan` on the host).

4.8 Secrets ownership rules

- Every secret has an **owner** (`created_by`) and a **purpose**; the creator owns it; only the owner (or an admin) lists, deletes and binds it (`core/security.py::owns_secret`, `require_binding`).
- Generated SSH keys are personal: **one key per user per host set**, named `<user>-<purpose>` (the UI's default is fine). A shared "team key" defeats the audit trail.
- Reserved names (`smtp_password`, `ai_default_api_key`, `github_token`, `system:*`) are platform secrets created by an admin through Admin settings.
- Legacy rows (`created_by` NULL, created before 1.1.0) are admin-only; an admin re-creates them under a named owner when convenient and deletes the legacy row (host bindings are checked at create/patch time, not retroactively, so re-point the host first).
- Leaving user: their secrets stay bound to hosts and targets until an admin replaces them (§4.9). Deleting the user does not delete the secrets.

4.9 Periodic access review (quarterly)

Run by an admin, evidence kept with the organisation's ISMS records (ISO 27001 A.5.18, SOC 2 CC6.2/CC6.3).

- [ ] Export the user list: Users page, or `GET /api/users` (fields: `username`, `role`, `disabled`, `mfa_enabled`, `mfa_required`, `locked_until`, `failed_logins`, `last_login_at`, `password_changed_at`, `created_at`).
- [ ] For every account: still employed/engaged? role still the least needed? `admin` count ≥ 2 and each admin known by name?
- [ ] Every account whose role is in `mfa_required_roles` has `mfa_enabled: true`; chase the rest (they are restricted anyway).
- [ ] Accounts with no `login.ok` in 90 days (`GET /api/audit?event=login.ok&username=<u>&since=<90 d ago>`): disable.
- [ ] Export the quarter's audit trail: `GET /api/audit/export?since=<quarter start>` → store as C2 with the review.
- [ ] From the export, review: `user.updated` (role changes) and `registration.approved` — each has a ticket; `mfa.reset`, `user.password_reset`, `user.unlocked` — each has an identity check recorded; `secret.bound` by non-admins — expected hosts only; `host.created/updated` — hosts still exist and belong to the organisation.
- [ ] Secrets page (admin view): every row has an owner who is active; legacy NULL-owner rows re-created; unused rows deleted.

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

- [ ] Hosts page: every host still needed; `authorized_keys` on each host contains only the keys avdlab currently binds.
- [ ] Automation users (§4.6): owner confirmed, password age acceptable.
- [ ] Admin settings reviewed: `registration_enabled`, `mfa_required_roles`, `ai_shared_allowed`, `ai_allow_private_urls`.
- [ ] Record date, reviewer, findings, actions; file with the risk register review (03 §3.4).

4.10 Offboarding checklist

Day 0 (before or at the moment access ends):

- [ ] Disable the account (`PATCH /api/users/{id}` `{disabled: true}`) — every session dies immediately.
- [ ] If the person was an admin: confirm ≥ 2 other admins remain; rotate `AVDLAB_SESSION_SECRET` (07 §7.5) and the Fernet keys (07 §7.6) if they had shell access to the orchestrator or to `.env`.
- [ ] Replace secrets they created that are still bound: generate new SSH keys (owned by a current user), bind them to the hosts, remove the old public keys from each host's `~/.ssh/authorized_keys`, delete the old secret rows. Re-issue arista.com / CloudVision / NetBox / AI tokens they entered.
- [ ] Lab hosts: remove any personal account they had there; rotate the sudo password if one was bound.
- [ ] External: remove their access to the git remotes labs are pushed to, the GitHub repository (if a maintainer), CloudVision tenant, NetBox.
- [ ] Automation users they owned: reassign owner, rotate password.

Within 30 days:

- [ ] Delete the account once nothing of theirs is still bound (`DELETE /api/users/{id}`).
- [ ] Record the offboarding in the ISMS log with the audit rows (`user.updated … disabled`, `user.deleted`, `secret.deleted`) as evidence.

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

CHAPTER 5

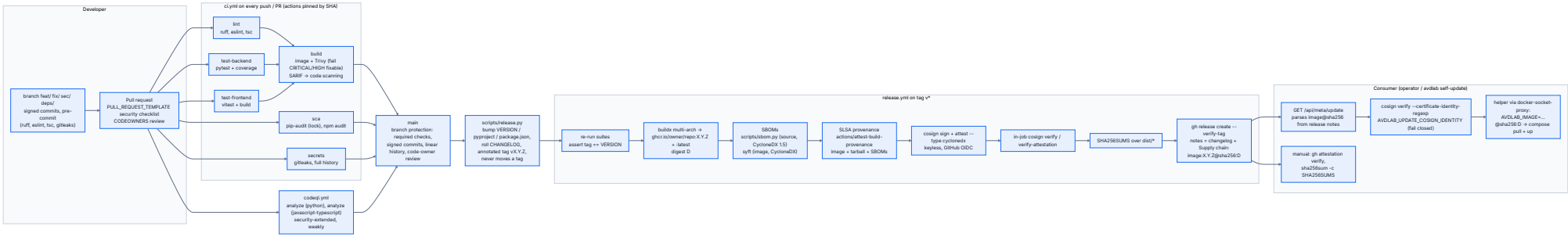
Secure development

SDLC as practised: reviews, static analysis, SCA, scanning, tests, release, provenance, suppliers.

How avdlab 1.1.0 is developed, tested, built and released, written as the assurance view of `CONTRIBUTING.md` and `docs/1.1/09-development.md`: for every practice the file that implements it, so that a reviewer mapping to NIST SSDF or ISO 27001 A.8.25–8.34 can check the evidence rather than take the description on trust. Where a practice is a GitHub *setting* rather than a file (branch protection), the chapter says so — settings are the repository owner's evidence.

5.1 Pipeline overview

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks



Release pipeline

5.2 Source control and change management

PRACTICE	IMPLEMENTATION	EVIDENCE
trunk	`main` is always releasable; releases are annotated tags `v<version>` on `main`	`CONTRIBUTING.md` → *Branching*
branches	short-lived `feat/`, `fix/`, `sec/`, `deps/`; rebase, squash-merge	same
review	one approving review from a code owner; security-sensitive paths (`backend/avdlab/core/`, `api/auth.py`, `deploy/`, `agent/`, `.github/`, `scripts/release.py`, `scripts/sbom.py`, `SECURITY.md`, both lockfiles) have explicit owners	`.github/CODEOWNERS`
PR template	*What*, *How to test*, general checklist (changelog, tests, lint, docs) and a **security checklist**: input validation, authZ, secrets, frontend hrefs, audit events, dependencies, deploy pinning, lab-host command quoting — "n/a is a valid answer, silence is not"	`.github/PULL_REQUEST_TEMPLATE.md`
required checks on `main`	`lint`, `test-backend`, `test-frontend`, `sca`, `secrets`, `build` (`ci.yml`), `analyze (python)`, `analyze (javascript-typescript)` (`codeql.yml`)	names in `CONTRIBUTING.md`, `docs/_changes/C.md` §(b); **the requirement itself is a GitHub branch-protection setting**
other protections	*Require signed commits*, *Require linear history*, *Require review from Code Owners*, *Dismiss stale approvals*	GitHub setting (recommended in `CONTRIBUTING.md`)
commit style	imperative, scoped; security fixes reference the GHSA/CVE; every user-visible change has a `CHANGELOG.md` line	`CONTRIBUTING.md`
one patch per fix	every shipped fix gets its own patch version so `GET /api/meta` says exactly what runs	`CONTRIBUTING.md`, `docs/1.1/08-operations.md` §8.5
local hooks	pre-commit: `check-yaml/json/toml`, merge-conflict, large files, `detect-private-key`, ruff (lint + fix), `ruff-format` (manual stage), gitleaks, eslint, tsc	`.pre-commit-config.yaml`

5.3 Static analysis

TOOL	SCOPE	RULES / CONFIGURATION	GATE
ruff	`backend/`, `scripts/`	`E`, `F`, `W` plus security-relevant bugbear/bandit rules: `B023` (loop variable binding), `B904` (`raise … from`), `S113` (requests without timeout), `S324` (weak hashes), `S501` (TLS verification disabled), `S602` / `S604` / `S605` (shell=True / `os.system`), `S608` (SQL from strings); `E501` ignored; `tests/**` may `E402`	`lint` job fails on findings; `ruff format --check` is **report-only** (`continue-on-error`) — a known formatting backlog of ~117 files
eslint	`frontend/`	project config (`npm run lint`)	`lint` job
tsc	`frontend/`	`tsc -b --noEmit` (`npm run typecheck`)	`lint` job
CodeQL	`python` (backend, scripts), `javascript-typescript` (frontend)	`security-extended` query suite; `paths-ignore` for `dist`, `node_modules`, the MSW worker and test files; on push to `main`, pull requests and weekly (Mondays 04:17 UTC)	required checks `analyze (…)`; findings in GitHub code scanning
Trivy	the built api image	`vuln,secret` scanners; CRITICAL/HIGH with `ignore-unfixed: true` fail the `build` job; second pass writes SARIF (CRITICAL/HIGH/MEDIUM) uploaded to code scanning (not for fork PRs)	`build` job

5.4 Software composition analysis and dependency pinning

ECOSYSTEM	MANIFEST	LOCK	INSTALL	AUDIT	UPDATE
Python	`backend/pyproject.toml` (floors; `pyavd==6.4.0`, `anta==1.10.0` exact)	`backend/requirements.lock` — **hash-pinned**, resolved for Python 3.11 / Linux (111 packages at 1.1.0)	`pip install --require-hashes --no-deps -r requirements.lock` (image, CI)	`pip-audit -r backend/requirements.lock --no-deps --strict` in `sca`; clean at 1.1.0	Dependabot weekly (`/backend`, minor+patch grouped, `pyavd` / `anta` ignored); refresh with `uv pip compile --python-version 3.11 --generate-hashes --no-strip-extras --output-file backend/requirements.lock backend/pyproject.toml` (pip-tools equivalent in the lock header)
npm	`frontend/package.json`	`frontend/package-lock.json`	`npm ci` only (the Dockerfile fails without the lock)	`npm audit --omit=dev --audit-level=high` gates; the full audit is report-only. At 1.1.0: 0 production advisories; 13 dev-only (vite 5 / vitest 2 / tailwind 3 transitive) whose fixes are major upgrades — tracked as a `deps/` follow-up	Dependabot weekly (`/frontend`, grouped)

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

ECOSYSTEM	MANIFEST	LOCK	INSTALL	AUDIT	UPDATE
container bases	`deploy/Dockerfile.backend`, `agent/Dockerfile`	every `FROM` is `tag@sha256:…` (`node:22-bookworm-slim`, `python:3.11-slim`, `docker:27-cli`; agent `avd/universal:python3.11-avd-v6.4.0@sha256:…`)	—	Trivy on the built image	Dependabot `docker` for `/deploy` and `/agent`; re resolve digests as the Dockerfile header explains
compose images	`deploy/docker-compose.yml`	`caddy:2@…`, `postgres:16-alpine@…`, `valkey/valkey:8.0-alpine@…`, `tecnativa/docker-socket-proxy:0.3.0@…`; `netboxcommunity/netbox:${NETBOX_VERSION}` stays a tag (operator-configurable; the resolved `v4.2` digest is in `docs/_changes/B.md`)	—	—	Dependabot `docker`
GitHub Actions	`.github/workflows/*.yml`	every `uses:` pinned to a **full commit SHA** with the version in a trailing comment	—	—	Dependabot `github-actions` group; resolve a SHA with `git ls-remote -tags … 'refs/tags/<tag>^{}'`
lab-host tooling	`AVDLAB_CLAB_VERSION` (0.79.0), `AVDLAB_CLAB_IMAGE` (`ghcr.io/srl-labs/clab:0.79.0`), `AVDLAB_AGENT_IMAGE`, `AVDLAB_ANTA_IMAGE` (`:1.10.0`)	containerlab installed from the pinned GitHub release with `sha256sum -c` against `checksums.txt`; Docker Engine from the signed `download.docker.com` repository (`hosts/recipes.py`)	bootstrap recipe	—	deliberate bump with a release note

Policy: a pin is changed only by a reviewed pull request; Dependabot PRs run the full CI; `pyavd` / `anta` bumps are a release decision validated with the lab playbook (`docs/04-lab-playbook.md`). `--no-deps` with hashes means a package not in the lock cannot enter the image.

5.5 Secret scanning

- **gitleaks** runs in CI (`secrets` job, full history `fetch-depth: 0`, `GITLEAKS_CONFIG: .gitleaks.toml`) and in pre-commit. An organisation-owned repository needs the `GITLEAKS_LICENSE` secret (`docs/_changes/C.md`).
- **Allow-list rationale** (`.gitleaks.toml`): `[extend] useDefault = true` keeps every default rule; the `[allowlist]` (`regexTarget = "match"`) excludes paths that are integrity hashes or build output (`package-lock.json`, `requirements.lock`, `dist/`, `node_modules/`, `.svg`, the MSW worker, screenshots) and **named fixture values with a comment each**: backend test credentials (`Blue-Lantern-42x`, `S3cret-smtp-Pa55`), the RFC 6238 example TOTP seed `JBSWY3DPEHPK3PXP`, MSW mock passwords, the AVD upstream example `sha512` hashes and IPsec `shared_key`s, lab-only defaults (`cvp123!`, `device_password … default: 'arista'`), and documentation placeholders (`change-me-session-secret`, `<token>`). A new fixture is added **with a comment**, never by disabling a rule (`CONTRIBUTING.md` → *Secrets in fixtures*).
- Trivy's `secret` scanner checks the built image as a second net.
- Real credentials never enter the repository, including tests and docs: `deploy/.env.example` ships every secret empty.

5.6 Tests

SUITE	COMMAND	1.1.0 RESULT	NOTES
backend	`cd backend && python -m pytest -q -p no:cacheprovider`	**354 passed, 8 skipped**	no Docker/SSH needed: SSH, Docker and the lab host are faked at single seams (`SSHClient.connect`, `images.docker.backend`, `http_client()`); security tests `tests/test_security_11.py` (31), platform `tests/test_platform_11.py` (28)
backend coverage	`--cov=avdlab --cov-report=xml` in CI	`coverage.xml` + `junit.xml` artefact (14 days), line rate in the step summary	—
frontend	`cd frontend && npm test` / `npm run test:coverage`	**176 passed** (20 files), **87.0 % lines**	`security11.test.tsx` (13): audit page, MFA-required handling, `safeHref`, upload replace
release tooling	`python3 -m pytest scripts/tests -q`	**6 passed**	`test_release.py`: tag immutability, clean-tree rule, SBOM/SHA256SUMS
lab playbook	`docs/04-lab-playbook.md`	manual	end-to-end against real cEOS before a `pyavd` / `anta` bump

Rule (`CONTRIBUTING.md`): new behaviour comes with a test next to the existing ones; a frontend feature that talks to the API also gets its MSW handler so `VITE_MOCK=1` covers the whole surface. Pure security functions are unit-tested directly: `role_denial`, `origin_allowed`, `TokenBucket`, `redact_text`, `password_violations`, `image_error`, `parse_jump`, `check_branch`.

5.7 Release process and verification

Cut (`scripts/release.py`, tested in `scripts/tests/test_release.py`): `release.py patch|minor|major|X.Y.Z` bumps `VERSION`, `backend/pyproject.toml`, `frontend/package.json`, renames `## [Unreleased]` → `## [X.Y.Z] - <date>`, commits `release X.Y.Z` and creates an **annotated** tag `vX.Y.Z`. Rules: an existing tag is **never moved** (*"tag vX exists — bump again"*); the working tree must be clean after the release commit (`--allow-dirty` to override, so nothing

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

uncommitted leaks into the tarball); `--tar DIR` also writes `avdlab-X.Y.Z.cdx.json` (CycloneDX 1.5 source SBOM from `scripts/sbom.py`: pypi purls with SHA-256 hashes from the lock, npm purls with SRI hashes, dev packages `scope: excluded`, root component with git SHA and licence) and `SHA256SUMS`.

Build (`.github/workflows/release.yml`, on `v*` tags, permissions `contents: write, packages: write, id-token: write, attestations: write`):

1. `test-backend` and `test-frontend` run again.
2. `tag == VERSION` is asserted; a lightweight tag only warns.
3. `docker/build-push-action` builds `linux/amd64,linux/arm64` and pushes `ghcr.io/<owner>/<repo>:X.Y.Z` and `:latest` with BuildKit `provenance: false, sbom: false` (so the manifest list has one predictable digest).
4. Source SBOM (`scripts/sbom.py`) and image SBOM (`anchore/sbom-action` → syft, CycloneDX JSON).
5. SLSA build provenance (`actions/attest-build-provenance`) for the image (`push-to-registry: true`) and for the tarball and both SBOMs.
6. `cosign sign --recursive` and `cosign attest --recursive --type cyclonedx` (keyless, GitHub OIDC), then an in-job `cosign verify` / `verify-attestation` with the exact identity `<server>/<owner>/<repo>/.github/workflows/release.yml@refs/tags/vX.Y.Z`.
7. `sha256sum -- * > SHA256SUMS` over `dist/`.
8. `gh release create --verify-tag` with the changelog section plus a *Supply chain* section carrying `<image>:X.Y.Z@sha256:<digest>` (the line `meta/update.py::digest_from_notes` parses) and the verification commands.

Verify as a consumer (replace `OWNER/REPO`, `X.Y.Z`, `DIGEST`):

```
# image signature: who built it (the release workflow of this repository, on this tag) and from which OIDC issuer
cosign verify \
  --certificate-identity https://github.com/OWNER/REPO/.github/workflows/release.yml@refs/tags/vX.Y.Z \
  --certificate-oidc-issuer https://token.actions.githubusercontent.com \
  ghcr.io/OWNER/REPO@sha256:DIGEST

# image SBOM attestation (CycloneDX), print the root component
cosign verify-attestation --type cyclonedx \
  --certificate-identity https://github.com/OWNER/REPO/.github/workflows/release.yml@refs/tags/vX.Y.Z \
  --certificate-oidc-issuer https://token.actions.githubusercontent.com \
  ghcr.io/OWNER/REPO@sha256:DIGEST | jq -r .payload | base64 -d | jq .predicate.metadata.component

# SLSA provenance (GitHub Attestations) for the image and for the source tarball
gh attestation verify oci://ghcr.io/OWNER/REPO@sha256:DIGEST --repo OWNER/REPO
gh attestation verify avdlab-X.Y.Z.tar.gz --repo OWNER/REPO

# asset integrity
sha256sum -c SHA256SUMS
```

Verify automatically (the running instance): `AVDLAB_UPDATE_SOURCE=github:OWNER/REPO`, `AVDLAB_UPDATE_MODE=image`, `AVDLAB_UPDATE_COSIGN_IDENTITY=^https://github.com/OWNER/REPO/.github/workflows/release.yml@refs/tags/v` and the cosign CLI in the image (`COPY --from=ghcr.io/sigstore/cosign/cosign:v2.4.1 /ko-app/cosign /usr/local/bin/cosign`). The update job then refuses an unpinned or unsigned image before touching `deploy/.env` (01 §1.9).

5.8 Vulnerability management

ITEM	COMMITMENT (`SECURITY.MD`)	MECHANISM
intake	private GitHub security advisory (preferred) or e-mail `dietrich@arista.com`, subject `avdlab security`	repository *Security* tab
acknowledgement	within **3 business days**, with a tracking reference and first assessment	maintainer
fix target	**30 days** high/critical, **90 days** other, from acknowledgement; longer with explanation	`sec/` branch → PR → patch release (one patch per fix)
disclosure	coordinated: advisory + changelog *Security* entry when the fix ships; credit unless declined	GitHub advisory, `CHANGELOG.md`
supported versions	latest **1.1.x** patch only; 1.0.x and earlier unsupported; pre-releases unsupported	`SECURITY.md` → *Supported versions*
dependency vulnerabilities	Dependabot weekly PRs; `pip-audit --strict` and `npm audit` block CI; Trivy blocks fixable CRITICAL/HIGH in the image; CodeQL weekly	`.github/dependabot.yml`, `ci.yml`, `codeql.yml`
triage of scanner output	code-scanning alerts (CodeQL, Trivy SARIF) reviewed by a code owner; a dismissed alert carries a reason	GitHub code scanning
operator notification	the About page / `GET /api/meta/update` shows the new version within `AVDLAB_UPDATE_CHECK_INTERVAL` (24 h); release notes name the advisory	`meta/update.py`
out of scope	third-party products (EOS, CloudVision, ACT, NetBox, containerlab, Docker, Caddy — routed upstream), findings requiring an admin or root, DoS by a legitimate operator on their own host, unproven scanner noise, mock mode	`SECURITY.md` → *Scope*

5.9 Third parties and suppliers

SUPPLIER / COMPONENT	ROLE IN AVDLAB	DATA SHARED	PINNED HOW	REVIEW CADENCE
Arista **AVD** (`pyavd`, `arista.avd` collection, universal image)	build pipeline in-process; ANTA runner and `cv_deploy` in the agent container	lab inputs (inside the deployment)	`pyavd==6.4.0`; agent image `tag@digest`	each AVD release, deliberate bump + playbook
Arista **ANTA**	validation, PSIRT scan	device credentials to switches, PSIRT inventory	`anta==1.10.0`; `ghcr.io/aristanetworks/anta:1.10.0`	with AVD

SUPPLIER / COMPONENT	ROLE IN AVDLAB	DATA SHARED	PINNED HOW	REVIEW CADENCE
Arista **cEOS-lab**	the switches	— (licensed download with the arista.com token)	version chosen per image	operator
containerlab (srl-labs)	topology runtime on lab hosts	—	release `0.79.0`, `sha256sum -c` against the release checksums; wrapper image `:0.79.0`	Dependabot does not cover it: bump with `AVDLAB_CLAB_VERSION`
Docker Engine / `docker:27-cli`	lab hosts, orchestrator, update helper	—	signed apt/dnf repository; helper `tag@digest`	Dependabot docker
Caddy	TLS edge	client IPs, TLS	`caddy:2@digest`	Dependabot docker
PostgreSQL, **Valkey**	optional database, NetBox dependencies	all tables / NetBox cache	`tag@digest`	Dependabot docker
NetBox (netbox-community)	optional DCIM sync	lab inventory	`netboxcommunity/netbox:${NETBOX_VERSION}` (tag, operator choice)	operator
AI providers (Anthropic, OpenAI-compatible incl. local)	optional designer	template descriptor, non-secret knobs, topology summary, chat; ACT password for ACT labs (R-08)	provider API; base URL SSRF-checked	operator reviews provider terms; admin opt-in for shared model
Let's Encrypt	certificates	domain name	Caddy ACME	—
GitHub (Actions, GHCR, Releases, Attestations, Dependabot, code scanning, advisories)	the whole SDLC and release distribution	source, images, SBOMs	actions by SHA; releases by digest + cosign	GitHub's own SOC reports are the supplier evidence
Sigstore (Fulcio/Rekor via cosign keyless)	signing and transparency	release digests and the workflow identity	`sigstore/cosign-installer` by SHA	—
PyPI / npm registries	dependencies	—	hashes / integrity in the lockfiles	Dependabot weekly
Python (`python:3.11-slim`), **Node** (`node:22-bookworm-slim`)	runtimes	—	`tag@digest`	Dependabot docker

Supplier review for the operating organisation (ISO 27001 A.5.19–5.21, SOC 2 CC9.2): record each row above that a deployment actually uses, with the contract or terms (SaaS: CloudVision, AI provider, Let's Encrypt, GitHub), the data class sent (02 §2.5), and the date reviewed.

5.10 Secure coding conventions

The conventions of `docs/1.1/09-development.md` §9.7 as rules a reviewer applies to every pull request:

RULE	ENFORCEMENT
Secrets never on argv. Transport is stdin (`sudo -S` , `curl -K -` , `docker run -i` , remote `ardl`), a 0600 file removed in `finally` (env-files, `GIT_ASKPASS` , `GIT_SSH_COMMAND -i`), or an HTTP header	review; `S602` / `S605` ruff rules on shell construction; the credential-travel matrix (01 §1.5) must be updated
`ctx.redact(...)` **at every reveal.** Any code path that calls `secret_store.reveal()` or derives a credential registers it before logging	review; `redact_text` tests; PR security checklist → *Secrets*
`shlex.quote` **on every interpolated shell argument;** validate before shell: `check_mgmt_ip` (ipaddress), `env_file_content` / `check_secret_value` (no CR/LF/NUL), `check_branch` , `parse_jump`	review; tests in `test_platform_11.py`
Every secret has a `purpose` **;** bindings go through `require_binding(user, secret, purposes, field)` ; admin bypass only there	review; `test_security_11.py`
Every mutation leaves an audit event with an actor (`audit.record(...)` or a `ROUTE_EVENTS` entry); `detail` never contains a body or a secret	review; PR checklist → *Audit*
Admin-only routes live under `/api/users` , `/api/admin` , `/api/audit` **and also use** `require_role("admin")` so RoleGuard and the router agree; everything else is operator-writable, viewer-readable	`role_denial` tests
External URLs are validated: `https://` for manifests, no private addresses for AI base URLs unless allowed, no local git remotes unless allowed, no `token` auth over `http://` / `git://` , no `-` -prefixed URLs	tests
Frontend hrefs built from server/admin data go through `safeHref()` / `<SafeLink>` **;** no `dangerouslySetInnerHTML` with remote content; `localStorage` holds no PII	`security11.test.tsx` ; PR checklist → *Frontend*
New columns go into `core/db._ADDED_COLUMNS` **;** settings are `AVDLAB_*` fields in `core/config.py` (admin-editable ones also in `core/app_settings.py`) and documented in `deploy/.env.example` and `docs/1.1/03` §3.5	review
Containers stay non-root, read-only, capability-less; images and actions pinned	PR checklist → *Deploy*; Trivy
Lab-host files get explicit modes (0600 files, 0700 lab dirs, 0750 root)	PR checklist → *Lab hosts*
Tests and MSW handlers accompany behaviour; fixtures that look like secrets are allow-listed in `.gitleaks.toml` with a comment	CI

5.11 Known SDLC gaps at 1.1.0

- `ruff format --check` is report-only (formatting backlog); a one-shot formatting PR is the follow-up.
- 13 `npm audit` advisories in dev-only dependencies (vite/vitest/tailwind majors) — no runtime exposure; tracked.

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

- gitleaks could not be executed locally when the configuration was written; the CI job is the authoritative run.
- Branch protection, signed-commit enforcement and code-owner review are GitHub settings: the repository owner should export them (repository → *Settings* → *Rules*) as evidence alongside this chapter.
- Templates are trusted admin content; there is no automated check that a template step does not print a secret.

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

CHAPTER 6

Logging & monitoring

Log inventory, redaction, retention, export, what to alert on.

What avdlab 1.1.0 records, how sensitive each log is, how long it is kept, how to get it off the orchestrator, and what an operator should watch. The product gives you an audit trail with an actor, redacted job logs and a health check; it does **not** ship a SIEM, alerting or tamper-evident storage — those are the operator's, and this chapter says how to wire them.

6.1 Log inventory

LOG	PRODUCED BY	CONTENT	SENSITIVITY (02)	REDACTION	RETENTION	WHERE / HOW TO READ
Audit trail (`audit_log` table)	`core/audit.py::record`, `AuditMiddleware`, routers	`at`, `username` (target), `actor` (who; NULL for anonymous events), `event`, `ip`, `user_agent` (≤ 512), `detail` (≤ 2000: `METHOD path key=value`, `by <admin>: role a→b`, never a body)	C5 + C2 (IPs, usernames)	by construction: no bodies, no secret values	`AVDLAB_AUDIT_RETENTION_DAYS` = 365 (0 = forever); pruned at startup and daily	*Admin › Audit log* (filters since/actor/event/usern 100 per page); `GET /api/audit` (`limit` ≤ 1000, `offset`)
Job logs	`jobs/engine.py::JobContext.log`	timestamped lines of every job (SSH commands, containerlab, ansible, pyavd), `job queued …`, tracebacks on failure, `[log truncated]`	C5; may contain lab hostnames, IPs, config fragments	`Redactor`: registered secrets → `***`; built-in patterns (Bearer, key=value, URL creds, AWS ids, PEM blocks)	`AVDLAB_JOB_LOG_RETENTION_DAYS` = 90 (directories removed; DB rows stay); cap `AVDLAB_JOB_LOG_MAX_BYTES` 20 MiB	`/data/jobs/<id>/log.` `GET /api/jobs/{id}/l` `WS /api/jobs/{id}/stream` job drawer
Application stdout	uvicorn access log, `avdlab.*` loggers (`avdlab.security`, `avdlab.audit`, `avdlab.terminal`, `avdlab.meta.update`, …)	request lines (method, path, status — no cookies), startup warnings (generated session secret path, bootstrap admin password **once**, weak-password warnings), key validation errors, housekeeping summaries (`pruned N audit rows`, `job housekeeping: removed N job directories`), terminal open/close/errors, update progress	C5; **the one-time bootstrap password** (C1) when `AVDLAB_PUBLIC_URL` is set	none beyond what the code logs	Docker logging driver (default `json-file`, unbounded unless configured)	`docker compose logs`
Update log	`meta/update.py`	`update.log` (helper output, cosign verify line), `update.json` `{from, to, mode, at, job_id, helper, image, ok}`, `update-check.json`	C5	—	overwritten per update	`/data/update.log`; `G` `/api/meta/update` → `last_update`

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

LOG	PRODUCED BY	CONTENT	SENSITIVITY (02)	REDACTION	RETENTION	WHERE / HOW TO READ
Caddy (profile `public`)	Caddy process	process log on stdout; **no access log by default** (the Caddyfile has no `log` directive)	C5 + C2 (client IPs if enabled)	—	Docker logging driver	`docker compose --pro public logs caddy`
Container logs of `db`, `netbox*`, `docker-socket-proxy`	the images	Postgres/NetBox process logs; proxy at `LOG_LEVEL: warning` (denied Docker API calls appear here)	C5	—	Docker logging driver	`docker compose logs <service>`
Host monitor samples (`host_metrics`)	`hosts/monitor.py` (20 s, hosts with a deployed lab or viewed in the last 10 minutes)	CPU, memory, disk, containers per host	C5	—	7 days (`RETENTION`)	Dashboard *Lab hosts* pa host Monitor page
Feedback	users	title, body, diagnostics (secrets masked), optional GitHub issue URL	C5 + C2	diagnostics masked	until deleted (`feedback.deleted` audited)	`/feedback`, `/data/feedback/<id>.`
Lab-host side	sshd, sudo, Docker, containerlab on each lab host	every avdlab SSH login, every `sudo` command, container starts	C5	—	the lab host's journald/syslog	`journalctl -u ssh`, `journalctl _COMM=sud`

6.2 Audit events

Grouped by what they tell an operator (`event` names as stored; `actor` is the acting username where one exists):

GROUP	EVENTS	TYPICAL `DETAIL`
sign-in	`login.ok`, `login.failed`, `login.locked` (attempt while locked), `login.lockout` (threshold reached), `login.mfa_pending`, `mfa.failed`	IP and user agent; `actor` NULL for failures
sessions	`logout.all` (count revoked), `session.revoked`	`sid`
MFA	`mfa.enabled`, `mfa.disabled`, `mfa.recovery_regenerated`, `mfa.reset` (admin)	`by <admin>`
passwords	`password.changed`, `password.change_failed`, `user.password_reset` (admin)	—
users	`user.bootstrap` (first admin, with the weak-password note), `user.created`, `user.updated` (`role a→b`, `disabled`, `enabled`), `user.deleted`, `user.unlocked`	`by <admin>: …`
registration	`registration.requested`, `registration.duplicate` (taken/pending username — enumeration attempt or mistake), `registration.approved`, `registration.rejected`	—
secrets	`secret.created`, `secret.deleted`, `secret.bound` (host or git binding)	name, kind, purpose, binding field
hosts	`host.created`, `host.updated`, `host.deleted`	host name / fields
labs	`lab.created` (also import), `lab.deleted`, `lab.file_written` (`PUT /labs/{id}/files/{path}`), `lab.git_configured`, `lab.git_removed`	`METHOD path lab_id=… path=…`
jobs	`job.cancelled`	job id, type
settings	`settings.updated`, `settings.ai_updated`, `settings.test_mail`	changed keys (no values)
AI	`ai.link.saved`, `ai.link.removed`, `ai.apply` (a proposal applied to a lab)	lab id, op count
feedback	`feedback.deleted`	id

Not audited as events (visible elsewhere): job submissions (the `jobs` table with `params`), reads, secret *reveals* inside jobs (the job log shows the step, never the value), update apply (`update.json` + the `app.update` job row), admin audit queries themselves.

6.3 What to alert on

Thresholds are starting points for a deployment with a handful of users; tune after a month of baseline.

SIGNAL	SOURCE	QUERY / PATTERN	WHY	SUGGESTED THRESHOLD
lockouts	audit	`event=login.lockout`	brute force or a forgotten password	any for an admin; ≥ 3 distinct users / hour
failed sign-ins from one IP	audit	`login.failed` grouped by `ip`	credential stuffing	≥ 20 / hour per IP (the token bucket turns the rest into 429s, which are **not** audited — see next row)
429 bursts	app stdout (uvicorn access log)	`"POST /api/auth/login HTTP/1.1" 429`	rate limiter engaged → attack in progress	≥ 50 / 10 min

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

SIGNAL	SOURCE	QUERY / PATTERN	WHY	SUGGESTED THRESHOLD
origin denied	app stdout	`403` on mutating `/api` with body `origin not allowed` (log the status; the body is not in the access line)	CSRF attempt or a misconfigured proxy/CORS	any sustained
`mfa_required` 403s for an admin	app stdout / `GET /users` `mfa_required: true`	—	an admin without MFA exists	any admin after onboarding week
registration duplicates	audit	`registration.duplicate`	username enumeration	≥ 10 / hour per IP
privilege changes	audit	`user.updated` with `role`, `user.created` with role admin, `registration.approved` role admin	every one needs a ticket	any
credential resets	audit	`user.password_reset`, `mfa.reset`, `user.unlocked`	identity-check evidence required	any
secret bound	audit	`secret.bound`	a credential now reaches a host/remote	any by a non-admin to a host not owned by that team
host changes	audit	`host.created`, `host.updated` (address, jump host, secrets)	new root-equivalent target	any
lab deletions, file writes	audit	`lab.deleted`, `lab.file_written`	destructive / content changes	review weekly
update applied	`update.json`, `app.update` job, app stdout `[update]` lines	`ok: false`, `cosign verification … failed`, `helper exited with`	failed or refused update	any
cosign failure	job log of `app.update` / `update.log`	`cosign verification of <image> failed`	unsigned or wrongly signed image offered	any — treat as incident until explained (07 §7.9)
`image_error`	`GET /api/meta/update`	non-null	release without a digest-pinned image	any
unreadable secrets	`python -m avdlab.cli check-keys` (schedule weekly)	exit 2 / `NO key opens N row(s)`	key misconfiguration before it bites	any
job log truncation / queue	app stdout / job logs	`[log truncated]`, `job queued: 4 of 4 job slots busy`	runaway job or capacity	repeated
housekeeping	app stdout	`pruned N audit rows`, `job housekeeping: removed N job directories`	confirms retention is running (export first)	absence for > 2 days
health	Docker `HEALTHCHECK`, `docker compose ps`	`unhealthy`	api down	any
certificates	Caddy log	renewal errors	expiry	any
socket proxy denials	`docker-socket-proxy` log	`403` lines	the api asked for a Docker endpoint it may not use (BUILD/EXEC)	any

6.4 Export and forwarding

Audit trail — scheduled NDJSON export. The export is an authenticated admin call; use a dedicated `admin` automation user (04 §4.6) **only if** `mfa_required_roles` excludes it — otherwise run the export from an operator's workstation after an interactive sign-in. Example with a cookie jar refreshed by a sign-in step:

```
#!/bin/sh
# /etc/cron.weekly/avdlab-audit-export — keep the audit trail beyond AVDLAB_AUDIT_RETENTION_DAYS and off the host
set -eu
URL=https://avdlab.example.com
JAR=$(mktemp); trap 'rm -f "$JAR"' EXIT
curl -fsS -c "$JAR" -H 'Content-Type: application/json' \
  -d "{\"username\":\"audit-export\",\"password\":\"$(cat /etc/avdlab/audit-export.pw)\"}" "$URL/api/auth/login" >/dev/null
SINCE=$(date -u -d '8 days ago' +%FT%TZ)
curl -fsS -b "$JAR" "$URL/api/audit/export?since=$SINCE" \
  | age -r age1examplepublickey... > /var/backups/avdlab/audit-$(date -u +%F).ndjson.age
curl -fsS -b "$JAR" -X POST "$URL/api/auth/logout" >/dev/null
# ship /var/backups/avdlab to the log platform (rsync, s3 cp, az storage blob upload, …)
```

The export carries IPs and usernames (C2): encrypt before it leaves the host, and store it where admins of avdlab cannot alter it (that is what makes the trail reviewable, R-17).

Container logs — Docker logging driver. In `deploy/docker-compose.yml` (or a `docker-compose.override.yml` so the shipped file stays untouched) set a driver with rotation or forwarding for `api` (and `caddy`):

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

```
services:
  api:
    logging:
      driver: json-file
      options: { max-size: "50m", max-file: "10" }
# or forward:
#   logging:
#     driver: syslog
#     options:
#       syslog-address: "tcp+tls://logs.example.com:6514"
#       tag: "avdlab-api"
```

`journald`, `fluentd`, `gelf` and `awslogs` drivers work the same way. The api logs to stdout only (read-only root filesystem), so the driver is the single collection point.

Job logs. Files under `/data/jobs/<id>/` — include `/data/jobs` in the backup (08) if 90 days is too short, or raise `AVDLAB_JOB_LOG_RETENTION_DAYS`. A log shipper that tails `/data/jobs/*/log.txt` on the host works but sees the same redacted content as the API.

Lab hosts. Forward `auth.log` / `journald` from each lab host; avdlab's SSH logins and every `sudo` appear there with the SSH user, which is the independent record of what the orchestrator did on the host.

6.5 Review cadence

CADENCE	WHO	WHAT
continuous	alerting (§6.3)	lockouts, 429 bursts, cosign failures, update failures, health
daily (public deployments)	operator on duty	*Admin › Audit log* with *since = 24h*: `login.*`, `registration.*`, `user.*`
weekly	an admin	privilege and credential events (`user.updated`, `*.reset`, `secret.bound`, `host.*`), `image_error`, `check-keys`, proxy denials; confirm the housekeeping lines and the export job ran
monthly	operator + security	`docker compose logs api` for warnings; Dependabot/code-scanning alerts (05 §5.8); job log retention adequate
quarterly	admin	access review with the quarter's export (04 §4.9); restore test includes a check that the audit export is readable (08 §8.7)
per release	operator	`update.json` `ok: true`, `GET /api/meta/build` matches the release, `docker compose config` digests

6.6 Clock synchronisation

Every timestamp — `audit_log.at`, `sessions.expires_at` (the 12-hour absolute cap), `totp_last_step`, job log lines, Fernet tokens (which embed a timestamp), cosign/Rekor verification and TLS validity — is taken from the orchestrator host's clock through the container. TOTP verification tolerates the usual ±1 step (30 s) but a host that drifts minutes locks every MFA user out and skews the audit trail. Run `chrony` / `systemd-timesyncd` (or the cloud provider's time source) on the orchestrator **and every lab host** (cEOS onboarding to CloudVision and certificate checks depend on it too), and record the time source in the deployment record (09). Audit rows are stored in UTC and exported in ISO 8601 with offset; correlate with other systems in UTC.

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

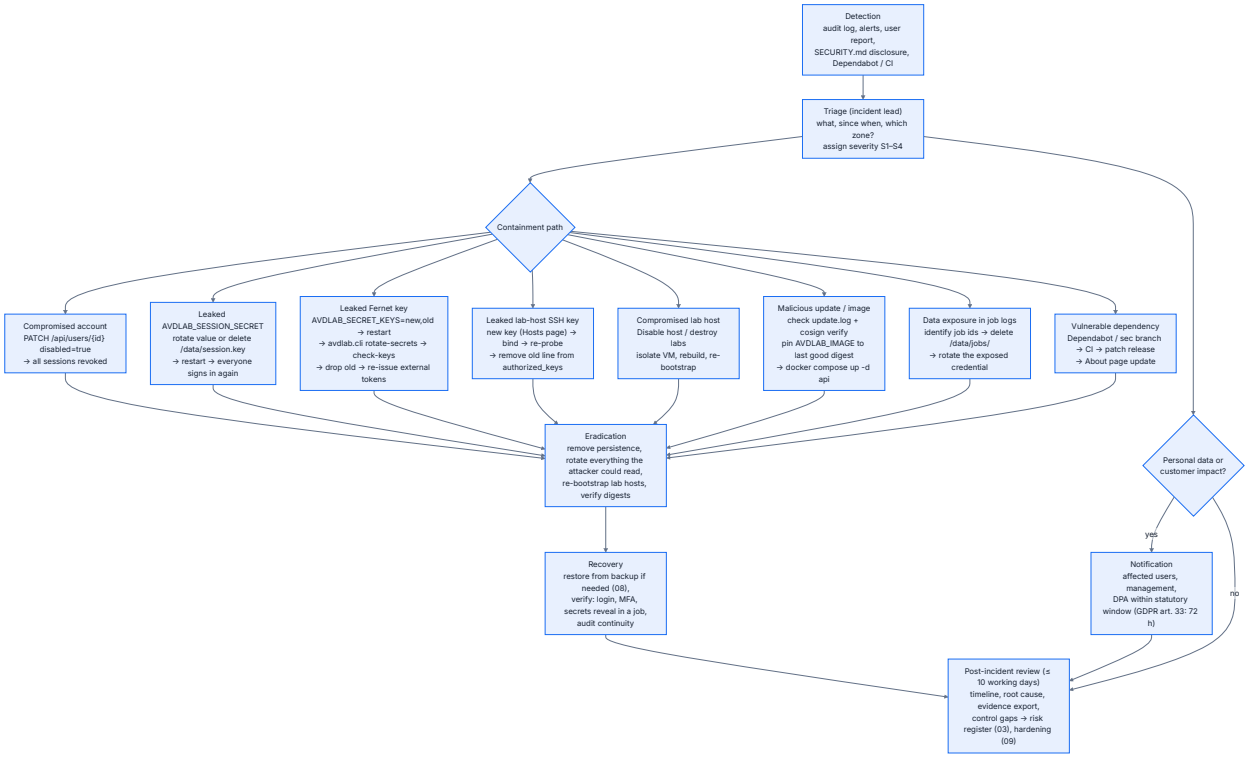
CHAPTER 7

Incident response

Playbook: severities, triage, containment, rotation procedures, recovery, review.

A playbook for the organisation operating avdlab 1.1.0. It assumes the roles of §7.1 are filled by named people, the backups of chapter 08 exist, and the monitoring of chapter 06 is what detects most incidents. Every scenario lists exact commands; `docker compose …` is run in `deploy/` on the orchestrator, API calls use an admin cookie jar (`curl -b jar`; sign in first with `POST /api/auth/login` and, with MFA, `POST /api/auth/mfa/verify`). Replace `https://avdlab.example.com` with your URL.

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks



Incident flow

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

7.1 Roles

ROLE	WHO (FILL IN)	DOES
Incident lead	an avdlab admin on duty	declares the incident, assigns severity, runs the playbook, keeps the timeline
Second admin	another admin	executes containment that the lead cannot do to themselves (disable, reset), independent check of every destructive step
Orchestrator host owner	whoever has root/`docker` on the orchestrator	`.env` changes, restarts, backups, host forensics
Lab host owner	per host	isolation, rebuild, `authorized_keys`, `journalctl`
Security contact	organisation's security function	notification decisions, DPA/legal, liaison with the maintainers via `SECURITY.md` when the cause is a product defect
Communications	management / team lead	tells users what to do (sign in again, re-enrol MFA)

Keep this table in the deployment record (09 §9.10) with phone numbers; the audit trail shows **who** acted, not how to reach them.

7.2 Severity

LEVEL	DEFINITION	EXAMPLES	RESPONSE
S1 critical	platform secrets or root on the orchestrator compromised; personal data of users exposed; malicious code running	Fernet key leaked, attacker with admin, tampered image deployed	immediate containment, all hands, notification assessment within 24 h
S2 high	one account or one lab host compromised; a credential that reaches third parties leaked	operator account phished, SSH key leaked, CloudVision token in a public repo	containment within 4 h
S3 medium	vulnerability known but not exploited; exposure limited to lab credentials or internal data	CVE in a dependency, device password in a public git remote, job log with an unredacted lab value	fix within the `SECURITY.md` windows (30/90 days) or configuration change within a week
S4 low	hygiene finding, no exposure	admin without MFA, update refused by `image_error`, legacy secret rows	next maintenance window

7.3 Triage (first 30 minutes)

1. **Freeze evidence first**, before changing anything that would alter it: `sh cd deploy docker compose logs --no-color --timestamps api > /secure/incident-$(date -u +%FT%H%MZ)-api.log curl -fsS -b jar "https://avdlab.example.com/api/audit/export?since=$(date -u -d '30 days ago' +%FT%TZ)" \ > /secure/incident-$(date -u +%F)-audit.ndjson docker run --rm -v avdlab_avdlab-data:/data -v /secure:/b alpine \ tar czf /b/incident-$(date -u +%F)-data.tgz -C /data . # includes jobs/, update.log, known_hosts` Store under restricted access (C1/C2 content). Note the exact clock offset of the host (`timedatectl`).
2. **Scope by zone** (01 §1.1): which zone is affected — an account (zone 0/2), the orchestrator (2–4), a lab host (5), a third party (6)? Which data classes (02)? Since when (`since` filter in the audit log; `sessions` rows via the affected user's Account page or the export)?
3. **Assign severity** (§7.2) and open the incident timeline (template §7.12).
4. **Decide notification need** early (§7.13): user PII or a customer's lab data involved?
5. Pick the containment scenario(s) below. Several usually apply together (an attacker with an admin session may have read `.env` through nothing in the product — but check the orchestrator host).

7.4 Scenario: compromised user or admin account

Indicators: `login.ok` from an unknown IP/user agent, `secret.bound` / `host.updated` the owner denies, sessions the user does not recognise on the Account page, `mfa.disabled` the user denies.

```
# 1. find the user id and disable (revokes every session immediately, audited user.updated "disabled")
curl -fsS -b jar https://avdlab.example.com/api/users | jq '.[] | select(.username=="alice") | .id'
curl -fsS -b jar -X PATCH -H 'Content-Type: application/json' -d '{"disabled": true}' \
  https://avdlab.example.com/api/users/<id>
# 2. what did the account do?  (actor = alice, last 30 days)
curl -fsS -b jar "https://avdlab.example.com/api/audit?actor=alice&since=$(date -u -d '30 days ago' +%FT%TZ)&limit=1000"
# 3. reset credentials before re-enabling: temporary password (forces change) and MFA re-enrolment
curl -fsS -b jar -X POST https://avdlab.example.com/api/users/<id>/reset-password     # returns temporary_password once
curl -fsS -b jar -X POST https://avdlab.example.com/api/users/<id>/mfa/reset
# 4. re-enable only after identity is confirmed out of band
curl -fsS -b jar -X PATCH -H 'Content-Type: application/json' -d '{"disabled": false}' https://avdlab.example.com/api/users/<id>
```

Then: delete secrets the account created during the window (`GET /api/secrets` as admin shows `created_by_name`) and rotate what they could have *used* — a bound SSH key reaches a host (§7.7), a token reaches a third party (§7.6 step 5). If the account was an **admin**: treat every secret's metadata and the full audit trail as read; rotate the session secret (§7.5) so no cookie the attacker saved survives; check `settings.updated` / `settings.ai_updated` for a changed AI endpoint or mail target; check `user.created` / `registration.approved` for a planted account; verify `AVDLAB_UPDATE_SOURCE` was not changed (it is env, so the orchestrator host would have to be compromised).

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

To sign the account out everywhere **without** disabling it (for example while you investigate): Users page → *Sign out*, or `POST /api/users/{id}/sessions/revoke` (admin; audit `user.sessions_revoked`). `POST /api/auth/logout-all` revokes only the caller's own sessions.

7.5 Scenario: leaked `AVDLAB_SESSION_SECRET`

Impact: anyone with the value can forge a cookie for any `sid` / `uid` / `sv` — but a forged cookie still needs a live `sessions` row, so the practical effect is session hijack, not account creation.

```
cd deploy
# explicit value in .env:
sed -i "s/^AVDLAB_SESSION_SECRET=.*/AVDLAB_SESSION_SECRET=$(openssl rand -hex 32)/" .env
# or generated file (AVDLAB_SESSION_SECRET empty): delete it, a new one is generated at start
docker compose run --rm --user 0 api rm -f /data/session.key
docker compose up -d api
docker compose logs api | grep -i "session secret"      # "generated a session secret into /data/session.key"
```

Every cookie is now invalid: all users sign in again (announce it). The old `sessions` rows stay until the daily pruner removes them but cannot be used without a cookie signed by the new secret. If the attacker may also have had an admin session, follow §7.4 for the accounts involved.

7.6 Scenario: leaked Fernet key (`AVDLAB_SECRET_KEYS` / `/data/secret.key`)

Impact: with the key **and** a copy of the database, every stored credential and every TOTP seed is readable; with the key alone, nothing (so check whether the database or a backup also leaked — the backup archive contains both unless the key was in `.env`).

```
cd deploy
# 1. generate a new key and put it FIRST, keep the old one so existing rows still decrypt
NEW=$(python3 -c "from cryptography.fernet import Fernet; print(Fernet.generate_key().decode())")
OLD=$(grep -E '^AVDLAB_SECRET_KEYS?=' .env | head -1 | cut -d= -f2-)      # or: docker compose exec api cat /data/secret.key
sed -i '/^AVDLAB_SECRET_KEYS\?=/d' .env; echo "AVDLAB_SECRET_KEYS=$NEW,$OLD" >> .env
docker compose up -d api                                                   # keys validated at startup
# 2. re-encrypt every secret and TOTP seed with the new primary key
docker compose exec api python -m avdlab.cli rotate-secrets --dry-run
docker compose exec api python -m avdlab.cli rotate-secrets
docker compose exec api python -m avdlab.cli check-keys                   # exit 0; "key #1 (primary): opens N row(s)"
# 3. drop the old key, restart, verify again (exit status 2 = some rows unreadable: put the old key back)
sed -i "s/^AVDLAB_SECRET_KEYS=.*/AVDLAB_SECRET_KEYS=$NEW/" .env
docker compose up -d api && docker compose exec api python -m avdlab.cli check-keys
# 4. the generated file, if it existed, is now a decrypt-only leftover: remove it
docker compose run --rm --user 0 api rm -f /data/secret.key
```

5. **Re-encryption does not un-leak the plaintext.** Assume every secret that existed before step 3 is known: re-issue the arista.com token, every CloudVision service-account and onboarding token, NetBox tokens, AI API keys (provider consoles), the SMTP password, the GitHub token; generate new SSH keys for every lab host (§7.7); change bound sudo/SSH passwords; ask every MFA user to **re-enrol** (admin `mfa/reset` per user, or let them disable and re-enable) because TOTP seeds were readable. Record each in the timeline.
6. Encrypted backups made before the rotation are still decryptable with the old key: keep them only as long as needed for forensics, under the stricter handling (08 §8.3).

7.7 Scenario: leaked lab-host SSH key or password

Impact: shell on the lab host as the avdlab user, `sudo` if NOPASSWD or the sudo password also leaked, `docker` group → root on that host.

```
# 1. generate a replacement key (owned by the acting admin, purpose ssh); the response carries the public key once
curl -fsS -b jar -X POST -H 'Content-Type: application/json' -d '{"name":"labhost1-2026-10","comment":"avdlab labhost1"}' \
  https://avdlab.example.com/api/hosts/keys/generate        # → {secret_id, public_key, …}
# 2. install the new public key on the host, then bind it and re-probe
ssh labuser@labhost1 'echo "<public_key>" >> ~/.ssh/authorized_keys'
curl -fsS -b jar -X PATCH -H 'Content-Type: application/json' -d '{"ssh_key_secret_id":"<secret_id>"}' \
  https://avdlab.example.com/api/hosts/<host_id>
curl -fsS -b jar -X POST https://avdlab.example.com/api/hosts/<host_id>/probe        # job must succeed
# 3. remove the old public key on EVERY host it was authorised on, and any unknown keys
ssh labuser@labhost1 'grep -v "<old key comment or fingerprint>" ~/.ssh/authorized_keys > ~/.ssh/ak && mv ~/.ssh/ak ~/.ssh/authorized_keys'
# 4. delete the old secret row (audited secret.deleted)
curl -fsS -b jar -X DELETE https://avdlab.example.com/api/secrets/<old_secret_id>
```

If a **password** (SSH or sudo) leaked: change it on the host (`passwd`), create a new `host`-purpose secret, bind it (`password_secret_id` / `sudo_password_secret_id`), delete the old row. Check the host for what the key was used for: `journalctl _COMM=sshd | grep labuser`, `journalctl _COMM=sudo`, `docker ps -a`, `last`. Unknown logins → treat as §7.8.

7.8 Scenario: compromised lab host

Containment: the lab host is root-equivalent for avdlab and may hold lab credentials (C3) and copies of CloudVision tokens in 0600 env-files during jobs.

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

1. Stop avdlab from using it: edit the host (Hosts page) to an unreachable address or remove its bound secrets (`PATCH /api/hosts/<id>` with `{"ssh_key_secret_id": null, "password_secret_id": null, "sudo_password_secret_id": null}`); `DELETE /api/hosts/<id>` (admin) only after labs are destroyed or re-pointed, since labs reference the host.
2. Isolate the VM/machine at the network (NSG, VLAN); snapshot the disk for forensics.
3. Assume disclosed: the SSH key/password bound to it (§7.7), any CloudVision/NetBox token used in a job on it during the window (re-issue), the device passwords of labs deployed there (change the knob, rebuild, redeploy), onboarding tokens written to switches (revoke in CloudVision).
4. Rebuild the host from a clean image; **Bootstrap** again from avdlab (signed Docker repo, pinned containerlab, `/opt/avdlab` 0750); re-add with a *new* key; verify the host-key fingerprint on first probe and compare with the old `known_hosts` line — remove the old line from `/data/known_hosts` first: `sh docker compose run --rm --user 0 api sh -c "grep -v '^labhost1' /data/known_hosts > /tmp/kh && cat /tmp/kh > /data/known_hosts"`
5. Check the orchestrator: a compromised lab host cannot reach the api unless the network allows it; confirm with the access log (06).

7.9 Scenario: malicious or unexpected self-update / image

Indicators: `GET /api/meta/update` shows a `latest` nobody released; `update.log` with a `cosign verification … failed` line; `last_update.ok: false` ; `GET /api/meta/build` shows an unexpected `git_sha` ; the About page version changed without a change ticket.

```
cd deploy
# 1. what ran?
docker compose exec api cat /data/update.json; docker compose exec api cat /data/update.log
docker compose exec api sh -c 'curl -s http://127.0.0.1:8080/api/meta' ; docker image inspect --format '{{.RepoDigests}}' $(docker compose images -
q api)
# 2. verify the running image against the release (OWNER/REPO, version, digest from the release page)
cosign verify --certificate-identity https://github.com/OWNER/REPO/.github/workflows/release.yml@refs/tags/vX.Y.Z \
  --certificate-oidc-issuer https://token.actions.githubusercontent.com ghcr.io/OWNER/REPO@sha256:DIGEST
gh attestation verify oci://ghcr.io/OWNER/REPO@sha256:DIGEST --repo OWNER/REPO
# 3. roll back by pinning the last known-good digest and recreating the api
sed -i 's#^AVDLAB_IMAGE=.*#AVDLAB_IMAGE=ghcr.io/OWNER/REPO:X.Y.Z@sha256:KNOWNGOOD#' .env
docker compose pull api && docker compose up -d api
# 4. stop further self-updates until explained
echo 'AVDLAB_UPDATE_MODE=manual' >> .env && docker compose up -d api
docker compose --profile update stop docker-socket-proxy
```

Then: who triggered `app.update` (the job row and `job.cancelled` /admin actions in the audit trail); was `AVDLAB_UPDATE_SOURCE` or the GitHub repository/account compromised (contact the maintainers through `SECURITY.md`); if the image ran, treat the orchestrator as compromised (S1): rotate §7.5 and §7.6, restore the data volume from a backup taken before the update if integrity is in doubt (08 §8.6). Source-mode updates (`git pull` + `compose build`) have no signature: inspect `git -C .. log` for unexpected commits.

7.10 Scenario: suspected data exposure in job logs

Indicators: a user reports a value in a job log; a template or custom command echoed a credential; `redact_text` missed a format.

```
# 1. find the jobs (type, lab, time) and grep the logs on the volume
docker compose exec api sh -c 'grep -l "<fragment>" /data/jobs/*/log.txt'
# 2. remove the affected log files (the job rows and results stay; the drawer shows an empty log)
docker compose run --rm --user 0 api sh -c 'rm -rf /data/jobs/<id1> /data/jobs/<id2>'
# 3. shorten retention if the pattern is systemic, until fixed
# AVDLAB_JOB_LOG_RETENTION_DAYS=7 in .env; docker compose up -d api
```

Then: rotate the exposed credential (it was readable by every role, including viewers, for as long as the log existed), record who read it if possible (job-log reads are not audited — assume all users), fix the template or report the missed pattern to the maintainers (`SECURITY.md`) with the *shape* of the value, not the value. If the exposure reached a backup, handle the backup copies (08 §8.3).

7.11 Scenario: vulnerable dependency or product vulnerability

- **Dependency (Dependabot / `pip-audit` / `npm audit` / Trivy fails CI):** the maintainers merge the Dependabot PR or a `deps/` PR, run `uv pip compile … --generate-hashes` for Python, release a patch (05 §5.7). Operators see it on the About page within 24 h and apply it (image mode, digest-pinned, cosign if configured). Severity for the deployment: S3 unless the advisory is reachable pre-auth over the network (then S2, consider taking a public deployment behind a VPN until patched).
- **Product vulnerability reported to the maintainers:** `SECURITY.md` process (acknowledge ≤ 3 business days, fix ≤ 30/90 days, coordinated disclosure). Operators: apply the patch release; read the advisory for indicators of past exploitation and search the audit export and access log for them.
- **Interim mitigations available without code:** disable public registration (`registration_enabled=false`), require MFA for operators, `AVDLAB_UPDATE_MODE=manual` , stop the `update` profile, bind the api to `127.0.0.1` and reach it through an SSH tunnel, tighten `AVDLAB_CSP` , lower `AVDLAB_SESSION_ABSOLUTE_MAX_AGE` .

7.12 Eradication, recovery and post-incident review

Eradication checklist

- [ ] every credential the attacker could have read is rotated (§7.5–7.8 as applicable) — list them in the timeline

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

- [ ] planted accounts removed (`user.created`, `registration.approved` in the window), roles restored
- [ ] admin settings compared with the deployment record (AI endpoint, mail, `mfa_required_roles`, `registration_enabled`)
- [ ] `.env` compared with the last backup copy (`AVDLAB_UPDATE_SOURCE`, `AVDLAB_IMAGE`, `AVDLAB_DOCKER_HOST`, proxies)
- [ ] orchestrator host checked (OS accounts, `docker ps -a`, cron, `authorized_keys`) when zone 2–4 was involved
- [ ] lab hosts re-bootstrapped when zone 5 was involved; `known_hosts` lines reviewed
- [ ] running image digest matches a verified release

Recovery checklist

- [ ] if the database or volume integrity is in doubt: restore per 08 §8.6 from a backup **before** the compromise, then re-apply legitimate changes from the audit export
- [ ] smoke test: sign in with MFA; `GET /api/meta/build`; a probe job on one host succeeds (proves secrets decrypt and SSH works); a lab `status` job; audit rows are being written (`login.ok` for the test)
- [ ] users informed: sign in again (session secret), re-enrol MFA (Fernet key), new temporary passwords (resets)
- [ ] monitoring back to normal thresholds; incident alerts cleared with a reason

Post-incident review — within 10 working days, 1–2 pages, filed with the ISMS records:

```
Incident:        <id> — <one line>
Severity:        S<n>        Detected: <UTC>        Contained: <UTC>        Closed: <UTC>
Lead:            <name>      Participants: <names>
Timeline:        <UTC> event / action / by whom (from the audit export, logs, chat)
Scope:           zones <…>, data classes <…>, users affected <n>, lab hosts <…>, third parties <…>
Root cause:      <technical> / <procedural>
What worked:     <controls that detected or limited it>
What did not:    <gaps>
Actions:         <id> <action> <owner> <due>  → risk register rows updated (03), checklist items added (09),
                 maintainers notified via SECURITY.md (if product defect)
Evidence kept:   <paths of the frozen logs/exports, retention>
Notification:    <none | users on <date> | DPA on <date> ref <…> | customer <…>>
```

7.13 Notification considerations

- **Users:** any rotation that affects them (session secret, Fernet key, resets) is announced with what to do and why, without details that help an attacker still inside.
- **Personal data (GDPR, when the deployment has EU users):** avdlab stores usernames, e-mail addresses, full names, registration reasons, IP addresses and user agents (02 §2.2). If their confidentiality was breached and the breach is likely to result in a risk to the individuals, the **controller** (the operating organisation) notifies its supervisory authority within **72 hours** of becoming aware (art. 33) and the individuals without undue delay when the risk is high (art. 34); document the assessment even when you decide not to notify. Processors involved (SMTP provider, AI provider, cloud host) may have contractual notification duties toward you — check the agreements recorded in 05 §5.9.
- **Customers / lab owners:** lab content (C3/C4) that belongs to a customer engagement is notified under the engagement's terms.
- **Maintainers:** a product defect goes through `SECURITY.md` (private advisory or e-mail), with the version, mode and indicators; the maintainers coordinate disclosure and credit.
- **Third parties whose credentials leaked:** Arista (arista.com token, CloudVision), NetBox admins, the AI provider — revoke/re-issue through their consoles; some providers want to know about a leaked key.
- **Law enforcement / insurer:** per the organisation's policy; keep the frozen evidence (§7.3) in its original form.

7.14 Exercise

Once a year, walk through §7.5 (session secret) and §7.7 (SSH key) on a test instance or the real one in a maintenance window: both are non-destructive, take under 30 minutes, and prove that the people in §7.1 can find `.env`, the Hosts page, the audit export and the backups. Record the exercise like an incident (§7.12) — auditors ask for it (ISO 27001 A.5.24, SOC 2 CC7.4).

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

CHAPTER 8

Backup & recovery

What to back up, schedule, encryption, RPO/RTO, restore procedure and test.

avdlab 1.1.0 keeps all state in one Docker volume (`avdlab-data`, mounted at `/data`), optionally a Postgres volume, and a handful of values in `deploy/.env`. This chapter says what must be in a backup, how to take one that is useful (encrypted, off-host, restorable), how to restore, and what is lost when a key is lost. The product does not schedule backups; the operator does. `docs/1.1/03-installation.md` §3.4 is the short version for operators; this is the procedure an auditor expects (ISO 27001 A.8.13, SOC 2 A1.2–A1.3).

8.1 What to back up

ITEM	LOCATION	CLASS (02)	NEEDED TO…	WITHOUT IT
database	`/data/avdlab.db` (SQLite, default) **or** the `avdlab-pg` volume / `pg_dump` (profile `postgres`)	C1 (ciphertext), C2, C4, C5	restore users, sessions, hosts, labs metadata, jobs, targets, settings, encrypted secrets, audit trail	everything is gone except the lab files
Fernet key(s)	`/data/secret.key` (0600) **or** `AVDLAB_SECRET_KEYS` / `AVDLAB_SECRET_KEY` in `deploy/.env`	**C1**	decrypt every secret, TOTP seed and lab `secret_knobs`	every stored credential and every MFA enrolment is **unrecoverable** (§8.7)
session secret	`/data/session.key` or `AVDLAB_SESSION_SECRET`	C1	keep existing cookies valid	users sign in again once — harmless
`deploy/.env`	the deploy directory	C1	reproduce the configuration: keys, `AVDLAB_IMAGE` (the exact running digest), `POSTGRES_PASSWORD`, `NETBOX_*`, update source, proxies, limits	manual reconstruction; Postgres/NetBox data unreadable without their passwords
known hosts	`/data/known_hosts`, `/data/git_known_hosts`	C4	keep host-key pinning (protection against a swapped host)	TOFU again on first contact after restore
lab repositories	`/data/labs/<slug>/` (git repos with inputs, build outputs, ANTA runs, snapshots)	C4 (+C3)	the labs themselves	labs with a configured git remote can be re-imported from it; the rest are lost
job logs and artefacts	`/data/jobs/<id>/`	C5	history, troubleshooting, evidence	job rows and results survive in the database; logs are gone
audit export	wherever the weekly NDJSON export lands (06 §6.4)	C5 + C2	audit history beyond `AVDLAB_AUDIT_RETENTION_DAYS` and independent of the database	rows older than 365 days are pruned anyway; a database restore rewinds the trail to the backup time
cEOS images	`/data/images/`	C4 (licensed)	avoid re-downloading multi-hundred-MB tarballs	re-download with the arista.com token (which is a secret — see first rows)
feedback, update bookkeeping	`/data/feedback/`, `/data/update.json`, `update.log`, `update-check.json`	C5	completeness	cosmetic
Caddy	`caddy_data` volume (certificates, ACME account)	C1 (private keys)	avoid Let's Encrypt rate limits and a re-issue delay	Caddy re-issues; staged rate limits may bite after repeated restores
NetBox (profile `netbox`)	`netbox-pg`, `netbox-media`, `netbox-redis` volumes	C4	NetBox state	re-sync from labs
compose files	`deploy/docker-compose.yml`, `deploy/Caddyfile`, any `docker-compose.override.yml`	—	reproduce the stack	re-clone the release tag

Not needed: the api image (re-pulled by digest from `AVDLAB_IMAGE` or rebuilt from the release tag), templates (in the image), `/tmp` and `/home/avdlab` (tmpfs), the remote lab trees on lab hosts (derived; re-created by deploy/sync).

8.2 Consistency

SQLite is one file written by one process. A copy taken while the api writes may be torn. Three safe options, from simplest to most online:

1. **Stop–copy–start** (recommended for small deployments; seconds of downtime, no running jobs are interrupted if you wait for the Jobs page to be idle): `sh cd deploy && docker compose stop api docker run --rm -v avdlab_avdlab-data:/data -v "$PWD/../backup:/b" alpine \ tar czf /b/avdlab-data-$(date -u +%F).tgz -C /data . docker compose start api`
2. **SQLite online backup** without stopping (consistent snapshot of the database; copy the rest of the volume afterwards — the lab repos are git repositories and tolerate a non-atomic copy; a job that was writing a log loses its tail): `sh docker run --rm -v avdlab_avdlab-data:/data -v "$PWD/../backup:/b" alpine sh -c \ 'apk add --no-cache sqlite >/dev/null && sqlite3 /data/avdlab.db ".backup /b/avdlab.db" && \ tar czf /b/avdlab-data-files-$(date -u +%F).tgz -C /data --exclude=avdlab.db --exclude="avdlab.db-*" .'`

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

3. **Postgres**: `docker compose exec -T db pg_dump -U avdlab -Fc avdlab > backup/avdlab-$(date -u +%F).dump` while running (transactionally consistent), plus the volume copy for files.

Whatever the option, add `deploy/.env` (and the override file) to the same backup set. Note the running image digest (`grep AVDLAB_IMAGE deploy/.env`, or `docker compose images api`) so the restore pairs the data with the schema version that wrote it — avdlab migrates **forward** at startup (`create_all` + `_ADDED_COLUMNS`), never backward.

8.3 Encryption and storage requirement

A backup archive contains the ciphertexts **and** (unless the keys live in `.env`) the Fernet key, the session secret, the SQLite file with user PII, the lab repositories with device passwords, and job logs. Rules:

- **Encrypt before the archive leaves the host**, with a key that is *not* stored next to it. `age` with a recipient public key or `restic` with a repository password kept in the organisation's password manager both qualify; `gpg -e -r <key>` too. An unencrypted `.tgz` in a cloud bucket is a C1 breach waiting for a misconfigured ACL.
- **Separate custody**: whoever can read the backup bucket should not also hold the backup decryption key (and vice versa) — this is the only separation between "a copy of the data" and "every secret in plaintext".
- **Two copies, one off-host**: a local copy for fast restores and an off-host copy (object storage, another machine, Azure Backup of the disk for the VM case) for host loss. On Azure, Blob Storage with immutability (WORM) policies also addresses tampering.
- **Retention**: keep daily backups for 14 days, weekly for 3 months, one monthly for a year (adapt to the audit retention you need; the audit NDJSON exports can be kept longer independently). Delete expired backups — they are C1 material, and after a Fernet rotation the old archives remain decryptable with the old key (07 §7.6 step 6).
- **Access to the backup location is logged** by the storage system; review it in the quarterly check (§8.6).

8.4 Schedule example

A nightly cron on the orchestrator with `restic` to an S3/Azure/B2 bucket (restic encrypts client-side; the repository password is the custody boundary) and a weekly stop–copy consistency point:

```
# /etc/cron.d/avdlab-backup  (orchestrator host, user with docker access; restic installed)
# nightly 02:10 — online SQLite .backup + files + .env, encrypted by restic
10 2 * * *  root  /usr/local/sbin/avdlab-backup.sh nightly  >> /var/log/avdlab-backup.log 2>&1
# Sunday 03:10 — stop/copy/start for a fully consistent archive
10 3 * * 0  root  /usr/local/sbin/avdlab-backup.sh weekly   >> /var/log/avdlab-backup.log 2>&1
```

```
#!/bin/sh
# /usr/local/sbin/avdlab-backup.sh  nightly|weekly
set -eu
MODE=${1:-nightly}
DEPLOY=/opt/avdlab/deploy                  # where docker compose runs
STAGE=/var/backups/avdlab/stage; mkdir -p "$STAGE"; chmod 0700 /var/backups/avdlab "$STAGE"
export RESTIC_REPOSITORY=azure:avdlab-backups:/orchestrator-1     # or s3:… / b2:… / sftp:…
export RESTIC_PASSWORD_FILE=/etc/avdlab/restic.pw                # 0400 root; a second copy in the password manager
cd "$DEPLOY"
if [ "$MODE" = weekly ]; then docker compose stop api; fi
docker run --rm -v avdlab_avdlab-data:/data -v "$STAGE:/b" alpine sh -c '
  apk add --no-cache sqlite >/dev/null
  if [ -f /data/avdlab.db ]; then sqlite3 /data/avdlab.db ".backup /b/avdlab.db"; fi
  tar cf /b/data-files.tar -C /data --exclude=avdlab.db --exclude="avdlab.db-*" --exclude=images .'
if [ -n "$(docker compose ps -q db 2>/dev/null)" ]; then            # postgres profile running?
  docker compose exec -T db pg_dump -U avdlab -Fc avdlab > "$STAGE/avdlab-pg.dump"
fi
if [ "$MODE" = weekly ]; then docker compose start api; fi
cp .env "$STAGE/env"; [ -f docker-compose.override.yml ] && cp docker-compose.override.yml "$STAGE/"
grep -E '^AVDLAB_IMAGE=' .env > "$STAGE/image.txt" || docker compose images api > "$STAGE/image.txt"
restic backup --tag "avdlab-$MODE" "$STAGE"
restic forget --keep-daily 14 --keep-weekly 12 --keep-monthly 12 --prune
rm -rf "$STAGE"/*
restic snapshots --latest 1 --tag "avdlab-$MODE" | tail -n 3
```

`images/` is excluded to keep the archive small (re-downloadable); include it if the arista.com token is not readily available. For a Mac mini on a LAN the same script works with `RESTIC_REPOSITORY=sftp:nas:/avdlab` or a local `age`-encrypted tarball copied to a NAS; the requirement is the same: encrypted, off-host, with a tested restore.

8.5 RPO / RTO statement (template)

OBJECTIVE	VALUE (FILL IN)	HOW THE CONFIGURATION ABOVE MEETS IT
RPO — acceptable data loss	24 hours (nightly backup)	labs are also committed to git on every change and optionally pushed to a remote; audit exports weekly (06 §6.4) — a loss window of a day costs at most a day of job logs, audit rows, settings and new accounts
RTO — time to a working instance	2 hours	§8.6 takes ~30 minutes on a prepared host plus image pull; a cold host needs Docker, the release checkout and DNS (public profile)
Retention	14 daily / 12 weekly / 12 monthly	`restic forget` policy

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

OBJECTIVE	VALUE (FILL IN)	HOW THE CONFIGURATION ABOVE MEETS IT
Restore test	quarterly	§8.7 checklist, recorded
Dependencies	GHCR reachable (image by digest), Let's Encrypt (public), lab hosts untouched	an orchestrator restore does not affect running labs: containers on lab hosts keep running and are re-adopted by `lab.status` / `inspect`

8.6 Restore procedure

Prerequisites: a host with Docker and Docker Compose, the backup decryption key (restic password / age key) from its custodian, the release checkout matching `image.txt` (or at least the same `deploy/` files), and — for the public profile — DNS pointing at the new host.

1. **Prepare the deploy directory** on the target host: `sh git clone --branch vX.Y.Z https://github.com/OWNER/REPO.git /opt/avdlab && cd /opt/avdlab/deploy`
2. **Fetch the backup** into a private directory: `sh mkdir -p /root/restore && chmod 0700 /root/restore restic restore latest --tag avdlab-weekly --target /root/restore # or: age -d … | tar x … ls /root/restore/var/backups/avdlab/stage # avdlab.db, data-files.tar, env, image.txt, (avdlab-pg.dump)`
3. **Restore** `.env` and confirm the keys are present: `grep -E '^AVDLAB_(SECRET_KEYS?|SESSION_SECRET|IMAGE)=' .env`. If the Fernet key was the generated file, it is inside `data-files.tar` as `secret.key` — do not start the api before step 4, or it will generate a *new* key and every secret becomes unreadable (`check-keys` exit 2 tells you).
4. **Create and fill the volume** (fresh host: the volume is created empty by compose; name is `<project>_avdlab-data`, project `avdlab`): `sh docker volume create avdlab_avdlab-data docker run --rm -v avdlab_avdlab-data:/data -v /root/restore/var/backups/avdlab/stage:/b alpine sh -c \ 'tar xf /b/data-files.tar -C /data && cp /b/avdlab.db /data/avdlab.db && chown -R 10001:10001 /data && \ chmod 0600 /data/secret.key /data/session.key 2>/dev/null; ls -la /data'` Postgres instead: `docker compose --profile postgres up -d db`, wait for `healthy`, then `docker compose exec -T db pg_restore -U avdlab -d avdlab --clean --if-exists < …/avdlab-pg.dump`.
5. **Pin the image** the data was written by: `AVDLAB_IMAGE=` in `.env` from `image.txt` (a newer release also works — avdlab migrates forward — but restore first, upgrade second, so a failure is attributable).
6. **Start** the same profiles as before: `docker compose [--profile public] [--profile update] [--profile postgres] up -d`.
7. **Verify**: `sh docker compose logs api | grep -iE "generated|RuntimeError|Fernet|session secret" # must NOT say "generated a session secret"/"secret.key" for a restored volume docker compose exec api python -m avdlab.cli check-keys # exit 0, all rows on key #1 curl -fsS http://127.0.0.1:8080/api/meta | jq .version # or via Caddy` Sign in with an admin (MFA code must be accepted — proves the TOTP seeds decrypt and the clock is right); Hosts → *Probe* on one host (proves SSH secrets decrypt and `known_hosts` pinning still matches); open a lab → Files; *Admin › Audit log* shows history up to the backup time; `GET /api/meta/build` shows the expected version.
8. **Re-apply what happened after the backup** from the audit export and the lab git remotes (`lab.import_git`), inform users that sessions were reset (sessions in the restored database are older than 12 h and expired anyway).
9. **Re-enable backups** on the new host (§8.4) and update the deployment record (09 §9.10) with the new host.

8.7 Quarterly restore test checklist

Perform on a scratch host or VM, never on the production volume. Record date, tester, backup snapshot id, duration and findings; this record is the evidence auditors ask for.

- [ ] restic/age repository reachable; `restic check` passes; latest snapshot is less than 25 hours old
- [ ] decryption key obtained from its custodian through the documented process (and nowhere else)
- [ ] restore per §8.6 steps 1–6 completed; **time taken** recorded against the RTO (§8.5)
- [ ] `python -m avdlab.cli check-keys` → exit 0, zero unopenable rows
- [ ] admin sign-in **with MFA** succeeds
- [ ] a secret is usable: probe job on a host (or, isolated from production hosts, `GET /api/secrets` lists rows and a job that reveals one succeeds against a test host)
- [ ] a lab opens, its files and last ANTA report are present
- [ ] audit log readable up to the snapshot time; a stored NDJSON export from the same period opens and matches
- [ ] `.env` from the backup contains every secret the stack needs (`POSTGRES_PASSWORD`, `NETBOX_*` if used) and the running digest
- [ ] scratch instance destroyed afterwards (`docker compose down -v`), restore directory shredded (`shred -u` / `rm -P`), test record filed

8.8 Consequences of key loss

LOST	EFFECT	RECOVERY
all Fernet keys (`secret.key` / `AVDLAB_SECRET_KEYS`)	every `secrets` row, every TOTP seed and every lab `secret_knobs` value is unrecoverable; `check-keys` exits 2 with every row listed; jobs that need a credential fail at `reveal()` with `InvalidToken`	none for the data. Procedure: generate a new key, **delete** the unreadable rows (secrets: via the API; TOTP: admin `mfa/reset` per user, which clears the seed), re-enter every credential, re-bind hosts/targets, users re-enrol MFA, ACT labs get a new password knob. With the last admin locked behind MFA: `sqlite3 /data/avdlab.db "UPDATE users SET mfa_enabled=0, totp_secret=NULL, recovery_codes='[]', totp_last_step=NULL WHERE username='admin'"` with the api stopped, then sign in with the password and re-enrol
one key of several	rows encrypted with it are unreadable until found; others fine	restore the key from `.env` backup; `check-keys` shows how many rows per key
session secret	all cookies invalid	none needed; users sign in again

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

LOST	EFFECT	RECOVERY
`known_hosts`	TOFU pins lost	re-pinned on next connection — verify fingerprints manually for the first probe after the loss (R-10)
`.env`	configuration lost; with keys in `.env`, see the first row	reconstruct from `deploy/.env.example` + the deployment record; keys from the password manager copy
backup decryption key	every backup is unreadable	none; start a new backup chain immediately and store the new key with two custodians
the only admin's authenticator and recovery codes	administration locked out	another admin resets MFA; with a single admin, the stopped-api SQL above or (destroying all accounts) `DELETE FROM users` + restart re-bootstraps from `AVDLAB_ADMIN_*` (R-20)

Store the Fernet key (or `.env`) in **two** places with different custodians: the backup set and the organisation's password manager. A quarterly check that the password-manager copy equals the live key (`sha256sum` of both) costs a minute and prevents the first row of this table.

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

CHAPTER 9

Hardening checklist

Deployment checklist for LAN and public installs.

A tick-box checklist for deploying avdlab 1.1.0. Column **L** is a LAN deployment (Mac mini or any Docker host, users on a trusted network, `docker compose up -d`); column **P** is a public deployment (Azure VM or any internet-facing host, compose profile `public` with Caddy). ● = required, ○ = recommended, — = not applicable. Settings are `AVDLAB_*` variables in `deploy/.env` unless noted; the reference for each is `docs/1.1/03-installation.md` §3.5. Keep the completed checklist with the deployment record (§9.10) — it is the evidence that the defaults were applied.

9.1 Before you start

L	P	ITEM
●	●	[ ] A dedicated orchestrator host or VM, patched OS, automatic security updates on, time synchronised (`chronyc tracking` / `timedatectl`)
○	●	[ ] Disk encryption on the orchestrator (LUKS / Azure disk encryption / FileVault on a Mac mini) — the data volume holds the keys (R-05)
●	●	[ ] Docker Engine from the signed vendor repository; non-root users in the `docker` group are root-equivalent — only the operator account
●	●	[ ] A release checkout at a tag (`git clone --branch vX.Y.Z`) or the release tarball with `sha256sum -c SHA256SUMS` and `gh attestation verify` (05 §5.7)
●	●	[ ] `deploy/` readable by the operator account only: `chmod 0700 deploy && chmod 0600 deploy/.env`
●	●	[ ] Two admins identified; a password manager entry for the keys (08 §8.8)

9.2 Environment (`deploy/.env`)

L	P	ITEM
●	●	[ ] `AVDLAB_ADMIN_PASSWORD` is **not** `change-me` and meets the policy (12+ chars, 3 classes) — with `AVDLAB_PUBLIC_URL` set, a weak value makes the api log a random bootstrap password once; read it from `docker compose logs api` and change it at first sign-in
●	●	[ ] `AVDLAB_SESSION_SECRET` either empty (generated into `/data/session.key`) or `openssl rand -hex 32` ; never `change-me-session-secret` (the api refuses to start)
○	●	[ ] `AVDLAB_SECRET_KEYS=<Fernet key>` set explicitly (keeps the key out of the database backup; enables rotation) — or accept the generated `/data/secret.key` and back it up
●	●	[ ] `AVDLAB_SESSION_ABSOLUTE_MAX_AGE` left at 43200 (12 h) or shorter; `AVDLAB_AUTH_RATE_LIMIT` left at 20 (never 0)
○	●	[ ] `AVDLAB_MFA_REQUIRED_ROLES=["admin","operator"]` — operators are root on lab hosts (default is `["admin"]`)
●	●	[ ] `AVDLAB_AUDIT_RETENTION_DAYS` matches the organisation's log policy (365 default); export schedule planned (06 §6.4)
●	●	[ ] `AVDLAB_JOB_LOG_RETENTION_DAYS` (90), `AVDLAB_JOB_LOG_MAX_BYTES` (20 MiB), `AVDLAB_JOB_MAX_CONCURRENT` (4) reviewed
●	●	[ ] `AVDLAB_UPLOAD_MAX_BYTES` (4 GiB) and the Caddy `request_body` caps consistent if changed
●	●	[ ] `AVDLAB_GIT_ALLOW_LOCAL=false` , `AVDLAB_SMTP_ALLOW_INSECURE=false` , `AVDLAB_AI_ALLOW_PRIVATE_URLS=false` unless a documented reason (local model server on the LAN)
●	●	[ ] `AVDLAB_REGISTRATION_ENABLED` — `false` when accounts are created by admins only; if `true` , `AVDLAB_REGISTRATION_MAIL_TO` set so requests are seen
●	●	[ ] `AVDLAB_CORS_ORIGINS` contains only origins you serve (the defaults are the dev server and `localhost:8080` ; remove them on a public host)
●	●	[ ] `AVDLAB_FEEDBACK_MAIL_TO` / `AVDLAB_FEEDBACK_GITHUB_REPO` point at **your** addresses, not the shipped defaults, or feedback is disabled
●	●	[ ] No real secret is left in `.env.example` or committed; `.env` is in `.gitignore` of your deployment repo

9.3 Network exposure and profiles

L	P	ITEM
●	—	[ ] `AVDLAB_BIND_ADDRESS=0.0.0.0` only on a trusted LAN; otherwise `127.0.0.1` + VPN/SSH tunnel. Plain `http://` means cookies and passwords cross the LAN in clear (R-24) — prefer the `public` profile with an internal CA/DNS, or a VPN
—	●	[ ] `AVDLAB_BIND_ADDRESS=127.0.0.1` : the api is reachable only through Caddy
—	●	[ ] `AVDLAB_DOMAIN` , `AVDLAB_ACME_EMAIL` , `AVDLAB_PUBLIC_URL=https://<domain>` set; DNS A record exists **before** first start; `AVDLAB_ACME_STAGING` removed after the dry run
—	●	[ ] `AVDLAB_TRUST_PROXY_HEADERS=true` **and** `AVDLAB_TRUSTED_PROXIES` covers the Caddy address (defaults cover the compose network); never set it with the api published on a public address
—	●	[ ] Host firewall / NSG: 443 (and 80 for ACME redirect, 443/udp for HTTP/3) from the internet; 22 from admin addresses only; nothing else
●	●	[ ] Egress from the orchestrator limited to what is used: lab hosts (22), GitHub (443), GHCR, arista.com, CloudVision, NetBox, the AI provider, SMTP; blocks SSRF side effects (R-19)
●	●	[ ] `--profile update` started only when the About-page update or local image import is used; `AVDLAB_DOCKER_HOST=tcp://docker-socket-proxy:2375` ; `AVDLAB_PROXY_ALLOW_BUILD=0` unless source-mode updates are wanted; the commented `docker.sock` mount in `docker-compose.yml` stays commented

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

L	P	ITEM
●	●	[ ] `--profile postgres` : `POSTGRES_PASSWORD` set (`openssl rand -hex 24`), `AVDLAB_DATABASE_URL` uses it; the `db` port is not published
●	●	[ ] `--profile netbox` : all five `NETBOX_*` secrets set; `NETBOX_BIND_ADDRESS=127.0.0.1` unless the UI must be reachable; NetBox reached by the api as `http://netbox:8080` on the compose network
●	●	[ ] `docker compose config` run after editing: images are `@sha256` digest-pinned (except `netboxcommunity/netbox:${NETBOX_VERSION}` — pin it in `.env` if wanted)

9.4 Container hardening (verify, do not change)

L	P	ITEM
●	●	[ ] `docker compose config` shows for `api` : `user: 10001:10001` , `read_only: true` , `cap_drop: [ALL]` , `no-new-privileges:true` , `tmpfs` on `/tmp` and `/home/avdlab` , `mem_limit` , `cpus` , `pids_limit` , no `/var/run/docker.sock` volume
●	●	[ ] Existing volume from 1.0.x chowned once: `docker compose run --rm --user 0 api chown -R 10001:10001 /data`
●	●	[ ] `docker compose ps` shows `healthy` ; `docker compose logs api` shows no `RuntimeError` about keys or the session secret
○	●	[ ] Resource limits sized for the host (`AVDLAB_MEM_LIMIT` , `AVDLAB_CPUS` , `AVDLAB_PIDS_LIMIT` , `AVDLAB_TMP_SIZE`)

9.5 Caddy, TLS and browser headers

L	P	ITEM
—	●	[ ] `curl -sI https://<domain>` shows `strict-transport-security` , `x-content-type-options: nosniff` , `x-frame-options: DENY` , `referrer-policy` , `content-security-policy` (from the api), `permissions-policy` , and **no** `server` header
—	●	[ ] TLS grade checked with an external scanner (Caddy defaults are TLS 1.2+/modern ciphers); certificate renewal visible in `docker compose --profile public logs caddy`
●	●	[ ] CSP review when customising: `AVDLAB_INTRO_VIDEO_URL` on another origin needs `media-src` to include it (default `media-src 'self' https:` covers any https origin); an embedded video **iframe** is not allowed by the default CSP and should stay that way (`frame-src` is absent → falls back to `default-src 'self'`); a stricter deployment removes `https:` from `media-src` when the video is self-hosted
●	●	[ ] `AVDLAB_CSP` / `AVDLAB_PERMISSIONS_POLICY` not emptied unless Caddy sets equivalent headers
—	●	[ ] Caddy access logging enabled if client IP history is wanted (`log` directive in the site block; the shipped Caddyfile has none) and the Docker logging driver rotates it (06 §6.4)

9.6 Accounts, MFA and secrets

L	P	ITEM
●	●	[ ] First admin signed in, password changed, **MFA enrolled**, recovery codes stored in the password manager / safe
●	●	[ ] Second admin created (`POST /api/users` or Users page) and enrolled
●	●	[ ] `mfa_required_roles` verified on Admin settings (default `["admin"]` ; add `operator` for public deployments)
●	●	[ ] Every operator creates **their own** SSH key on the Hosts page (purpose `ssh`); no shared key; legacy NULL-owner secrets re-created with an owner
●	●	[ ] Platform secrets entered through the UI (encrypted): `smtp_password` , `ai_default_api_key` , `github_token` , arista.com token, CloudVision/NetBox tokens — not in `.env` where both are possible
●	●	[ ] Secrets list reviewed (admin view): every row has a known owner and purpose; nothing unused
●	●	[ ] AI designer: shared model off (`ai_shared_allowed=false`) unless the provider terms were reviewed; users told that lab data goes to the model; ACT labs use throw-away passwords (R-08)
●	●	[ ] `python -m avdlab.cli check-keys` → exit 0 (run inside: `docker compose exec api python -m avdlab.cli check-keys`)

9.7 Updates and supply chain

L	P	ITEM
●	●	[ ] `AVDLAB_UPDATE_SOURCE=github:<owner>/<repo>` points at the repository you trust (your fork if you build your own)
○	●	[ ] `AVDLAB_UPDATE_MODE=image` (digest-pinned releases) rather than `source` on a public host; or `manual` with a documented update procedure
○	●	[ ] `AVDLAB_UPDATE_COSIGN_IDENTITY=^https://github.com/<owner>/<repo>/.github/workflows/release.yml@refs/tags/v` and the cosign CLI in the image (`COPY --from=ghcr.io/sigstore/cosign/cosign:v2.4.1 /ko-app/cosign /usr/local/bin/cosign` in your Dockerfile build, or an image that ships it); `AVDLAB_UPDATE_REQUIRE_DIGEST=true` (default)
●	●	[ ] First update verified by hand once: `cosign verify …` / `gh attestation verify …` against the release page (05 §5.7)
●	●	[ ] After each update: About page version, `GET /api/meta/build` `git_sha` , `update.json` `ok: true`
●	●	[ ] Dependabot / release notifications watched for the repository (GitHub *Watch → Releases*)

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

9.8 Lab hosts

L	P	ITEM
●	●	[ ] Dedicated Linux VM or machine per lab host; nothing else runs on it; snapshot/rebuild path exists
●	●	[ ] A dedicated SSH user for avdlab (not root, not a personal account); key-based login; `PasswordAuthentication no` in `sshd_config` once keys work (beta Mac/Windows hosts excepted)
●	●	[ ] `sudo` scope: `NOPASSWD` only for what the recipe needs, or a sudo password bound as a `host` -purpose secret (04 §4.7); the `docker` group membership is understood as root-equivalent
●	●	[ ] Inbound SSH only from the orchestrator (and the jump host hop); the containerlab management bridge and cEOS mgmt addresses not routed off the host
●	●	[ ] Bootstrap run from avdlab (installs Docker from `download.docker.com`, containerlab `AVDLAB_CLAB_VERSION` with `sha256sum -c`, `/opt/avdlab` 0750); host-key fingerprint from the probe log (`[tofu] pinned …`) compared with `ssh-keyscan` on the host
●	●	[ ] Egress allow-list: `download.docker.com` , `github.com` (containerlab release), `ghcr.io` (agent, ANTA, clab, client images), CloudVision and arista.com if used
○	●	[ ] `auth.log` /journald forwarded (06 §6.4); time synchronised
●	●	[ ] No credential reused from another system; device passwords are lab-only values
○	○	[ ] For a jump host: the intermediate machine is also treated as a lab-host-class system (same key, same trust)

9.9 Backups, logging, monitoring

L	P	ITEM
●	●	[ ] Backup job installed (08 §8.4): nightly, encrypted, off-host; `.env` included; `restic snapshots` shows one
●	●	[ ] Backup decryption key stored with a custodian who is not the bucket owner (08 §8.3)
●	●	[ ] Restore test scheduled quarterly (08 §8.7) and a first restore done on a scratch host before go-live
●	●	[ ] Docker logging driver with rotation or forwarding for `api` (and `caddy`) (06 §6.4)
○	●	[ ] Weekly audit export job (06 §6.4) to storage the admins cannot alter
○	●	[ ] Alerts for lockouts, 429 bursts, cosign/update failures, health (06 §6.3)
●	●	[ ] NTP on the orchestrator and lab hosts (06 §6.6)

9.10 Deployment record

Keep one page per deployment (in the ISMS or the team wiki), updated at every change — it is what the incident lead reaches for first (07 §7.1):

```
Deployment:        <name>            Type: LAN | public          URL: <…>
Orchestrator:      <host/VM, OS, owner, location/region>        Docker: <version>
avdlab version:    <X.Y.Z>  image digest: <sha256:…>  update mode: <image|source|manual>  cosign identity: <…|none>
Profiles:          public | update | postgres | netbox
Key custody:       Fernet keys in <.env | /data/secret.key>; password-manager entry <…>; backup key custodian <name>
Admins:            <name> (MFA ✓), <name> (MFA ✓)        mfa_required_roles: <[…]>
Lab hosts:         <name, address, owner, VM/physical, sudo scope, bootstrap date>
External services: GitHub <repo>, arista.com <account>, CloudVision <tenant>, NetBox <url>, AI <provider/model, shared?>, SMTP <host>
Backups:           <repository, schedule, retention, last restore test date>
Logging:           <driver/destination>, audit export <schedule/destination>
Network:           inbound <rules>, egress <allow-list>, time source <…>
Checklist:         completed <date> by <name>; exceptions: <item → reason → compensating control → review date>
```

Every unticked ● item in the completed checklist is an **exception** recorded with a reason, a compensating control and a review date — that is what turns a checklist into audit evidence.

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

CHAPTER 10

Compliance matrix

NIST SSDF, ISO/IEC 27001:2022 Annex A and SOC 2 TSC mapped to evidence.

Three mappings for avdlab 1.1.0: the NIST Secure Software Development Framework (SSDF 1.1, SP 800-218), the technological and relevant organisational controls of ISO/IEC 27001:2022 Annex A, and the SOC 2 Trust Services Criteria. The status vocabulary is deliberate:

STATUS	MEANING
Implemented	the product or this repository does it; the evidence column names the file, workflow or document
Supported — operator procedure	the product provides the mechanism; the operating organisation must run the procedure and keep the evidence (this set tells it how)
Partial	implemented with a documented gap (see the notes and the risk register)
Outside the product	an organisational control (people, management, HR, legal) that no software supplies; the organisation owns it entirely
Not applicable	the control's subject does not exist in this system

A reminder from the README: the organisation is what gets certified or receives a SOC 2 report; these tables show which of its controls avdlab satisfies or supports, and where it must add its own.

10.1 NIST SSDF 1.1

PRACTICE	TASK(S)	HOW AVDLAB MEETS IT	EVIDENCE	GAPS / OPERATOR RESPONSIBILITY
PO.1 Define security requirements	PO.1.1–1.3	security requirements written for the code (secure-coding conventions, PR security checklist), the deployment (hardening defaults, checklist) and third-party components (pinning policy)	`docs/1.1/09-development.md` §9.7, `.github/PULL_REQUEST_TEMPLATE.md`, `CONTRIBUTING.md`, 09	requirements of the operating organisation's own policies to be layered on
PO.2 Roles and responsibilities	PO.2.1–2.3	code ownership for security-sensitive paths; vulnerability contact; maintainers' responsibilities in CONTRIBUTING	`.github/CODEOWNERS`, `SECURITY.md`, `CONTRIBUTING.md`	**training** of maintainers (PO.2.2) is outside the product; a single maintainer at 1.1.0 — the organisation decides whether a second owner is required
PO.3 Supporting toolchains	PO.3.1–3.3	toolchain specified and pinned: actions by SHA, images by digest, Python by hash, npm by lockfile; tools generate evidence (coverage artefacts, SARIF, SBOMs, attestations)	`ci.yml`, `codeql.yml`, `release.yml`, `Dockerfile.backend`, `requirements.lock`, `package-lock.json`	—
PO.4 Security check criteria	PO.4.1–4.2	gates: ruff security rules, eslint/tsc, pytest/vitest, pip-audit `--strict`, `npm audit --omit=dev --audit-level=high`, gitleaks, Trivy CRITICAL/HIGH fixable, CodeQL `security-extended`; results stored as artefacts and code-scanning alerts	`ci.yml` jobs `lint`, `test-*`, `sca`, `secrets`, `build`; `codeql.yml`	`ruff format` and the full `npm audit` are report-only (documented); branch protection making the checks *required* is a GitHub setting (05 §5.11)
PO.5 Secure environments	PO.5.1–5.2	CI with minimal `permissions`, `persist-credentials: false`, OIDC-issued short-lived tokens for signing/attestation; no long-lived signing key exists (keyless cosign)	workflows' `permissions` blocks, `sigstore/cosign-installer`, `attest-build-provenance`	GitHub-hosted runners: the organisation relies on GitHub's own controls (supplier)
PS.1 Protect code from unauthorised changes	PS.1.1	signed commits, code-owner review, linear history, required checks; tags never moved; `--verify-tag` on release	`CONTRIBUTING.md` → *Branching*, `scripts/release.py::tag_exists`, `release.yml`	branch protection is a setting; export it as evidence
PS.2 Verify release integrity	PS.2.1	`SHA256SUMS`, keyless cosign signature and CycloneDX attestation, SLSA provenance for image and assets, verification commands in the release notes; consumer-side digest + cosign check in the self-update	`release.yml`, `meta/update.py`, 05 §5.7	cosign verification on the instance is opt-in (`AVDLAB_UPDATE_COSIGN_IDENTITY`); operators should enable it
PS.3 Archive and protect each release	PS.3.1–3.2	immutable annotated tags, GHCR images by digest, release assets with SBOMs (source: `scripts/sbom.py`; image: syft)	GitHub Releases, `scripts/sbom.py`	the organisation keeps its own copy of the release artefacts it deployed if GitHub availability is a concern
PW.1 Design to meet requirements / threat model	PW.1.1–1.3	trust zones, boundary controls, risk register, by-design decisions documented	01, 03, `SECURITY.md` → *By design*	the register is re-rated by the organisation (03 §3.4)

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

PRACTICE	TASK(S)	HOW AVDLAB MEETS IT	EVIDENCE	GAPS / OPERATOR RESPONSIBILITY
PW.2 Review design	PW.2.1	design contracts per phase; architecture documentation reviewed with each release	`docs/CONTRACT-*.md`, `docs/_changes/A-C.md`, `docs/1.1/02-architecture.md`	an independent design review (outside the maintainers) is the organisation's decision
PW.4 Reuse well-secured software	PW.4.1, 4.4	pinned, audited third-party components; SBOM lists them; Dependabot proposes updates; suppliers table	05 §5.4, §5.9, `.github/dependabot.yml`	`pyavd` / `anta` excluded from Dependabot on purpose (deliberate bumps)
PW.5 Secure coding practices	PW.5.1	conventions enforced by review and lint: no secrets on argv, `ctx.redact`, `shlex.quote`, validation before shell, `safeHref`, purpose on secrets, audit on mutations	05 §5.10, ruff `S*` rules	—
PW.6 Compiler/interpreter/build security	PW.6.1–6.2	reproducible installs (`--require-hashes --no-deps`, `npm ci`), digest-pinned bases, multi-stage build, non-root runtime	`deploy/Dockerfile.backend`	—
PW.7 Code review	PW.7.1–7.2	human review by a code owner with a security checklist; automated review by CodeQL, ruff, eslint	PR template, `CODEOWNERS`, `codeql.yml`	with one maintainer, "review by another person" needs a second owner — organisation's call
PW.8 Testing	PW.8.1–8.2	354 backend, 176 frontend, 6 release-tooling tests; security-specific suites; coverage reported	05 §5.6, CI artefacts	no DAST/pen test in the pipeline; the organisation commissions penetration tests as its policy requires
PW.9 Secure defaults	PW.9.1–9.2	secure by default: MFA for admins, weak secrets refused at startup, digest-required updates, read-only non-root container, no default DB/NetBox passwords, SMTP TLS, SSRF guard, CSP; every relaxation is an explicit setting documented with its risk	`core/config.py`, `deploy/docker-compose.yml`, 09	LAN deployment over http is a documented trade-off (R-24)
RV.1 Identify vulnerabilities	RV.1.1–1.3	Dependabot weekly, pip-audit/npm audit/Trivy on every push, CodeQL weekly, private disclosure channel with acknowledgement ≤ 3 business days	`dependabot.yml`, `ci.yml`, `codeql.yml`, `SECURITY.md`	runtime monitoring of the deployment is the operator's (06)
RV.2 Assess, prioritise, remediate	RV.2.1–2.2	fix targets 30/90 days; one patch per fix; advisory + changelog entry; About page notifies instances	`SECURITY.md`, `CONTRIBUTING.md` → *One patch per fix*	operators must apply the patch (self-update or manual)
RV.3 Analyse root causes	RV.3.1–3.4	post-incident review template; risk register updated on findings; conventions extended	07 §7.12, 03 §3.4	the organisation runs the reviews and keeps the records

10.2 ISO/IEC 27001:2022 Annex A

CONTROL	TITLE	STATUS	EVIDENCE	NOTES
5.9	Inventory of information and other associated assets	Supported — operator procedure	02 §2.2 asset inventory; 09 §9.10 deployment record	the organisation assigns owners and adds the deployment to its asset register
5.12	Classification of information	Supported — operator procedure	02 §2.1 six classes with handling rules	adopt or map to the organisation's scheme
5.15	Access control	Implemented	`core/security.py::role_denial`, `require_role`; 04 §4.1	three roles; single tenant (R-12)
5.16	Identity management	Implemented / Supported	`api/users.py`, `api/registration.py`; 04 §4.2 lifecycle	identity verification of requesters is a procedure
5.17	Authentication information	Implemented	argon2id, password policy, TOTP with replay guard, recovery codes, `must_change_password`, uniform login errors, rate limit	`core/passwords.py`, `api/auth.py`, `core/ratelimit.py`
5.18	Access rights	Supported — operator procedure	Users page, `GET /api/users`, `GET /api/audit/export`; 04 §4.9 quarterly review, §4.10 offboarding	the review must be performed and recorded
5.19–5.21	Supplier relationships, agreements, ICT supply chain	Partial / Supported	05 §5.9 supplier table; pinning and verification (05 §5.4, §5.7); SBOMs	agreements with SaaS suppliers (CloudVision, AI provider, GitHub) are the organisation's
5.24	Incident management planning and preparation	Supported — operator procedure	07 playbook, roles, severity	staffing and annual exercise (07 §7.14) by the organisation
5.25	Assessment and decision on information security events	Supported	07 §7.2–7.3 triage and severity; 06 §6.3 signals	—
5.26	Response to information security incidents	Supported	07 §7.4–7.11 scenario commands	—
5.27	Learning from incidents	Supported	07 §7.12 review template → 03 register	—
5.28	Collection of evidence	Supported	07 §7.3 evidence freeze (logs, audit NDJSON, volume archive); audit rows with actor/IP/UA	audit log is not tamper-evident (R-17): forward exports

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

CONTROL	TITLE	STATUS	EVIDENCE	NOTES
5.37	Documented operating procedures	Implemented (documentation)	`docs/1.1/` (install, operate, update, rotate), this set	—
6.3	Information security awareness, education and training	Outside the product	—	maintainers' and operators' training is organisational
8.2	Privileged access rights	Implemented / Supported	`admin` role; MFA required for admins by default; last-admin guards; audited admin actions with `actor`; 04 §4.1, §4.4	lab hosts: sudo scope is the operator's (04 §4.7)
8.3	Information access restriction	Implemented	role model; secret scoping (`created_by` , `purpose` , reserved names); write-only secrets; admin-only audit	labs shared between operators by design
8.4	Access to source code	Implemented (repository)	`CODEOWNERS` , branch protection, signed commits	GitHub settings as evidence
8.5	Secure authentication	Implemented	server-side sessions (12 h absolute, revocable), MFA policy, rate limiting, lockout, `Secure` /HttpOnly/ `SameSite` cookie, OriginGuard	—
8.8	Management of technical vulnerabilities	Implemented (product) / Supported (deployment)	`SECURITY.md` SLAs, Dependabot, pip-audit, npm audit, Trivy, CodeQL; About-page update notice	operators apply patches; third-party products upstream
8.9	Configuration management	Implemented / Supported	hardened compose and Dockerfile defaults; every setting in `core/config.py` with documented default (`docs/1.1/03` §3.5); 09 checklist; `.env` in the backup	drift detection is the operator's (`docker compose config` per release)
8.10	Information deletion	Implemented / Supported	retention settings (audit, job logs, metrics, sessions); delete endpoints; 02 §2.6	hand deletion for some data; backups retained separately
8.13	Information backup	Supported — operator procedure	08: what, schedule, encryption, restore, quarterly test	**not performed by the product**
8.15	Logging	Implemented	`audit_log` with actor/target/IP/UA, redacted job logs, app stdout; retention; export (06 §6.1)	forwarding is the operator's
8.16	Monitoring activities	Supported — operator procedure	06 §6.3 alert catalogue, §6.5 cadence; health check; `check-keys`	no built-in alerting
8.19	Installation of software on operational systems	Implemented	self-update: https only, digest-pinned, optional cosign, admin-only, through the socket proxy; lab-host bootstrap from signed repos with checksum-verified containerlab	source mode unsigned (documented)
8.20	Networks security	Supported	bind to `127.0.0.1` behind Caddy; compose network isolation; SSRF guard; 09 §9.3 firewall/egress	network controls are the operator's
8.24	Use of cryptography	Implemented	Fernet/MultiFernet with rotation (`avdlab.cli`), argon2id, TLS via Caddy, SSH with pinned host keys, keyless signing; key custody documented (08 §8.8)	no HSM/KMS (R-05); volume encryption is the operator's
8.25	Secure development life cycle	Implemented	05; `CONTRIBUTING.md`	—
8.26	Application security requirements	Implemented	PR security checklist; conventions (05 §5.10)	—
8.27	Secure system architecture and engineering principles	Implemented	01 (zones, least privilege, defence in depth, secure defaults, fail closed)	—
8.28	Secure coding	Implemented	ruff security rules, CodeQL, conventions, `safeHref` , validation seams	—
8.29	Security testing in development and acceptance	Implemented	security-specific test suites, SCA, container and secret scanning in CI	no DAST/pen test in pipeline
8.30	Outsourced development	Not applicable	—	no outsourced development at 1.1.0
8.31	Separation of development, test and production environments	Implemented / Supported	mock mode (`VITE_MOCK=1`) and test fakes need no production systems; CI builds are never pushed except from `v*` tags	the operator keeps a test instance for update verification (`CONTRIBUTING.md` step 4)
8.32	Change management	Implemented	pull requests, required checks, code-owner review, changelog per change, one patch per fix, annotated immutable tags	—
8.33	Test information	Implemented	fixtures are synthetic and allow-listed (`.gitleaks.toml`); no production data in tests	—
8.34	Protection of information systems during audit testing	Supported	read-only API access for auditors via `viewer` ; audit export; mock mode for UI walkthroughs	—

Organisational controls not listed (5.1–5.8, 5.10–5.11, 5.13–5.14, 5.22–5.23, 5.29–5.36, 6.1–6.2, 6.4–6.8, 7.x physical) are **outside the product** and the operating organisation's responsibility in full.

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

10.3 SOC 2 Trust Services Criteria

CRITERION	FOCUS	STATUS	EVIDENCE	NOTES
CC1.x	Control environment (integrity, oversight, structure, competence, accountability)	Outside the product	—	the service organisation's
CC2.x	Communication and information	Partial	`SECURITY.md`, `CONTRIBUTING.md`, this set, release notes communicate security commitments and changes	internal communication is organisational
CC3.x	Risk assessment	Supported	03 risk register and treatment template	the organisation performs and records it
CC4.x	Monitoring activities (of controls)	Supported	04 §4.9 quarterly access review, 08 §8.7 restore test, 06 §6.5 cadence	performance and records are organisational
CC5.x	Control activities	Supported	the procedures across this set	—
CC6.1	Logical access security software, infrastructure, architectures	Implemented	role model, MFA policy, server-side sessions, OriginGuard, CSP; secret store with scoping; container hardening; 01	—
CC6.2	Registration and authorisation of users	Implemented / Supported	account requests with admin approval, roles, audited `registration.*` / `user.*`; 04 §4.2	identity verification procedure
CC6.3	Removal / modification of access	Implemented / Supported	disable revokes sessions; role changes audited; 04 §4.9–4.10 review and offboarding	periodic review performed by the organisation
CC6.4	Physical access	Outside the product	—	orchestrator and lab hosts' premises / cloud provider
CC6.5	Disposal of assets	Supported	02 §2.6 deletion; 08 §8.3 backup retention and deletion	disk sanitisation is organisational
CC6.6	Threats from outside system boundaries	Implemented / Supported	Caddy TLS + HSTS, rate limits, lockout, uniform errors, SSRF guard, `127.0.0.1` binding, firewall checklist (09 §9.3)	network controls operator's
CC6.7	Transmission and movement of information	Implemented	TLS, SSH with pinned keys, credentials via stdin/0600/headers never argv, redaction, SMTP TLS enforced; 01 §1.5 matrix	LAN http is a documented exception (R-24)
CC6.8	Prevention/detection of unauthorised or malicious software	Implemented / Supported	digest-pinned images, cosign-verifiable releases, Trivy, read-only container, socket proxy allow-list, signed Docker repo on lab hosts	host AV/EDR is the operator's
CC7.1	Detection of configuration changes and vulnerabilities	Implemented / Supported	CI scanners, Dependabot, CodeQL; `docker compose config` digests; `check-keys`; `image_error`	configuration drift monitoring on the host is operational
CC7.2	Monitoring for anomalies	Supported — operator procedure	audit trail with actor, export, 06 §6.3 alert catalogue	no built-in alerting/SIEM
CC7.3	Evaluation of security events	Supported	07 §7.2–7.3	—
CC7.4	Incident response	Supported	07 scenarios with commands; annual exercise	—
CC7.5	Recovery from incidents	Supported	07 §7.12, 08 restore	—
CC8.1	Change management (authorise, design, develop, test, approve, implement)	Implemented	PR workflow with required checks and code-owner approval; release pipeline; immutable tags; changelog; self-update with digest/signature gating	GitHub settings as evidence
CC9.1	Risk mitigation / business disruption	Supported	08 RPO/RTO template, backups; running labs survive an orchestrator outage	—
CC9.2	Vendor and business partner risk	Supported	05 §5.9 supplier table	agreements and reviews are organisational
A1.1–A1.3 (Availability)	Capacity, environmental protections, backups and recovery testing	Partial / Supported	resource limits, health check, job queue (A1.1); backups and restore test (A1.2–A1.3, chapter 08); single-node design, no HA	the organisation decides whether a single node meets its availability commitment
C1.1–C1.2 (Confidentiality)	Identification and disposal of confidential information	Supported	02 classification and deletion; encryption at rest for C1; redaction	—
PI1.x (Processing integrity)	—	Partial	pyavd validation before writes (`422` with violations), config hashes recorded, ANTA validation of deployed state	relevant only if the organisation commits to processing integrity for lab outputs
P1–P8 (Privacy)	—	Supported (minimum)	data minimisation in registration, retention settings, admin-only PII, 02 §2.6 data-subject handling, 07 §7.13 breach notification	privacy notices, consent and DPIAs are the controller's

10.4 Using the matrix

- For an **ISO 27001 statement of applicability**: copy the rows of §10.2 with status *Implemented* or *Supported* as the technical justification for the deployment's in-scope controls; add the organisation's own measures for every *Outside the product* row.
- For a **SOC 2 system description**: avdlab appears as a component of the in-scope system; §10.3 lists which controls it carries and which the organisation performs around it. Evidence of *operation* (not design) is what the auditor samples: access reviews, backup tests, incident records, change tickets — the artefacts this set tells you to keep.

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

- For an **SSDF attestation** (for example a customer's secure-software self-attestation form): §10.1 gives the practice-by-practice answer with evidence paths; the honest exceptions are the report-only formatter, dev-only npm advisories, the opt-in instance-side signature check, and the single-maintainer review model.
- Re-check the matrix at every minor release; a status change needs a changelog line and an update of the chapter that holds the evidence.

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks



avdlab 1.1.0 — documentation generated 22 October 2026 from `docs/1.1/` and the application source.
© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks. Arista, EOS, CloudVision and AVD are trademarks of Arista Networks. containerlab, ANTA and NetBox belong to their respective projects.